



Oil Refinery Plant

This presentation is a part of AnyLogic Standard Training Program



© The AnyLogic Company | www.anylogic.com

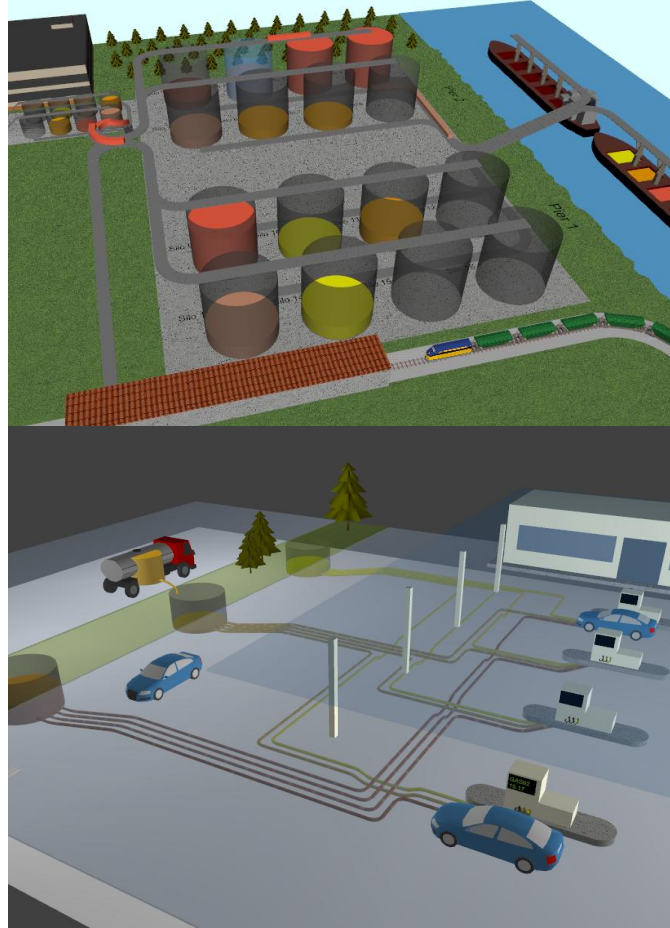
Oil Refinery Plant

- Let us build a simulation model of the oil refinery plant, which distills incoming crude oil into fractions.
- The model simulates the main processes of an oil refinery plant. Crude oil is transported by train, stored in the storage tank and is processed in the distillation tower. The distillation tower produces four types of petroleum products:
 - Liquid petroleum gas (LPG)
 - Gasoline
 - Diesel oil
 - Fuel oil
- Petroleum products are transferred to storage tanks where they remain until being dispatched to customers.
- During the model execution, we will calculate the number of production interruptions at the oil refinery plant. We will then configure and run an optimization experiment to minimize this number.

Oil Refinery Plant. Phase 1

- In this phase, we will create a flowchart using the Fluid Library blocks to define the process of the crude oil entering the system, being stored in the storage tank, transferred to the distillation tower and leaving the system afterwards. We will also animate these processes.

Fluid Library



- Allows you to model storage and transfer of fluids, bulk matter or large amounts of discrete items, which you do not want to model as separate objects.
- Provides interoperability with the Process Modeling Library and other AnyLogic libraries.
- The library engine maximizes the flow throughout the system. Flow rates in the library are piecewise-constant (constant within time intervals) and only change instantly at discrete moments of time. The execution speed of the Fluid Library models is therefore a lot higher than that of the System Dynamics models.

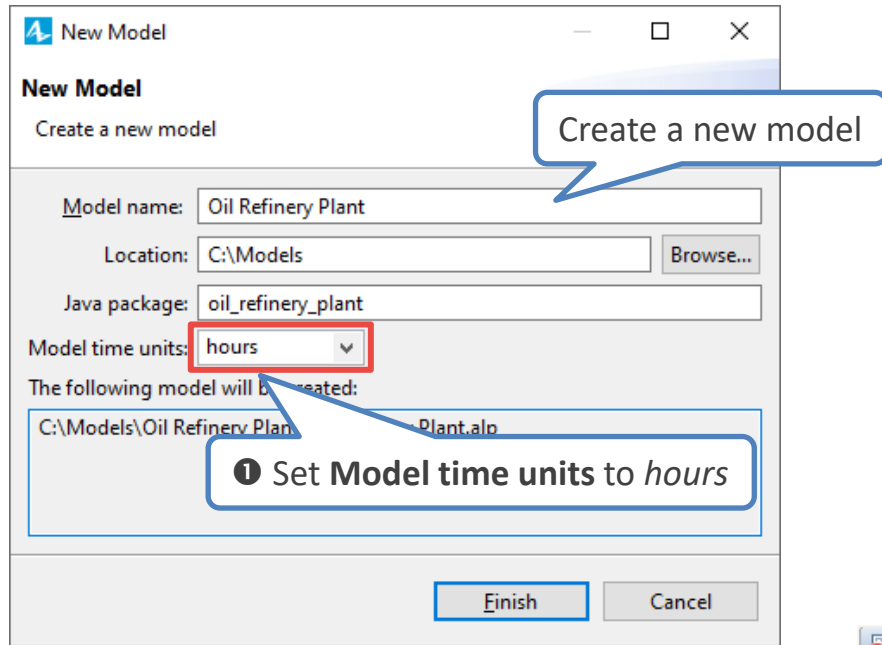
Fluid Library Blocks (1/2)

Block name	Icon in graphical editor	Description
<i>Fluid Source</i>		Generates the flow; is a typical starting block of a Fluid Library flowchart.
<i>Fluid Dispose</i>		Accepts the incoming flow and disposes it (removes from the system). Is a typical end block of a Fluid Library flowchart.
<i>Tank</i>		A finite capacity container for fluid or bulk matter. Can serve as a source of fluid or as a disposal container with finite capacity.
<i>Process Tank</i>		Accumulates fluid up to the Capacity level, optionally delays it for a given amount of time, and lets it flow out.
<i>Mix Tank</i>		Makes a mix of fluids coming in from up to five different sources, optionally delays (e.g. processes) the completed mix for a given amount of time, and lets it flow out.
<i>Pipeline</i>		Transports fluid from one point to another.
<i>Valve</i>		Restricts the flow rate or completely blocks the flow.
<i>Bulk Conveyor</i>		Conveys bulk or condensable matter from one point to another. Allows gaps and sections of different “density”.
<i>Fluid Convert</i>		The output rate of this block equals the input rate with a given “conversion factor,” which can amplify or diminish the flow.

Fluid Library Blocks (2/2)

Block name	Icon in graphical editor	Description
<i>Fluid Dropoff</i>		Drops off fluid from the agents passing through.
<i>Fluid Pickup</i>		Lets the agents passing through pick up fluid or bulk matter.
<i>Agent To Fluid</i>		Converts agents (discrete items) into the flow.
<i>Fluid To Agent</i>		Converts portions of fluid or bulk matter into agents (discrete items).
<i>Fluid Select Input</i>		Routes the flow from one of the two inputs to the output.
<i>Fluid Select Output</i>		Routes the flow from the input to one of the outputs.
<i>Fluid Split</i>		Splits the input stream into two different streams; the sum of output flow rates equals the input rate.
<i>Fluid Merge</i>		Merges two streams of flow into one; the output flow rate equals the sum of the input rates.
<i>Fluid Exit</i>		Forwards the flow into the connected FluidEnter block.
<i>Fluid Enter</i>		Accepts the flow from the connected FluidExit block.

Oil Refinery Plant. Phase 1. Step 1



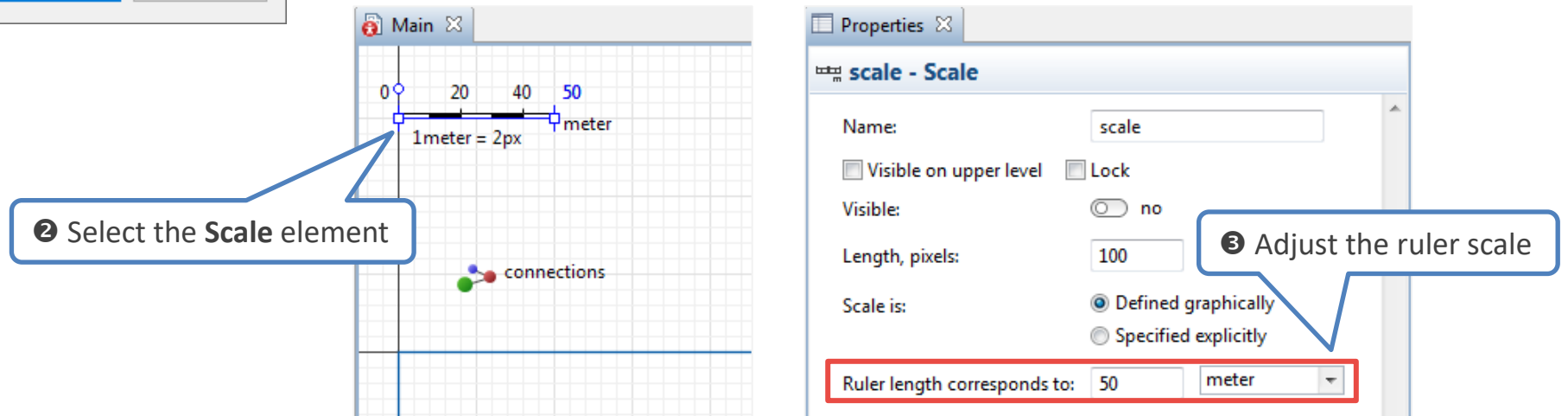
Create a new model and name it *Oil Refinery Plant*.

❶ Set **Model time units** to **hours**.

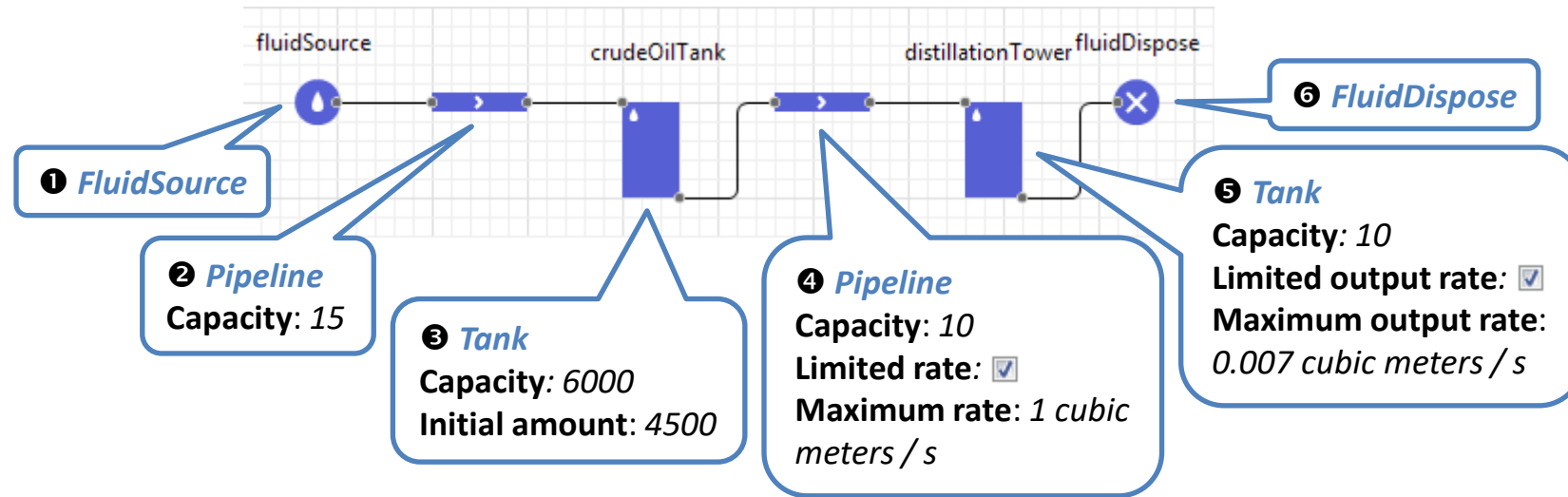
Click **Finish**. The *Main* agent diagram will open.

❷ Select the **Scale** element located above the visible area marked by the blue rectangle. To find the element, pan the graphical editor by pressing the right mouse button and moving the mouse without releasing the mouse button.

❸ In the **Properties** of the **Scale** element, set the **Ruler length** corresponds to property value to *50 meters*.



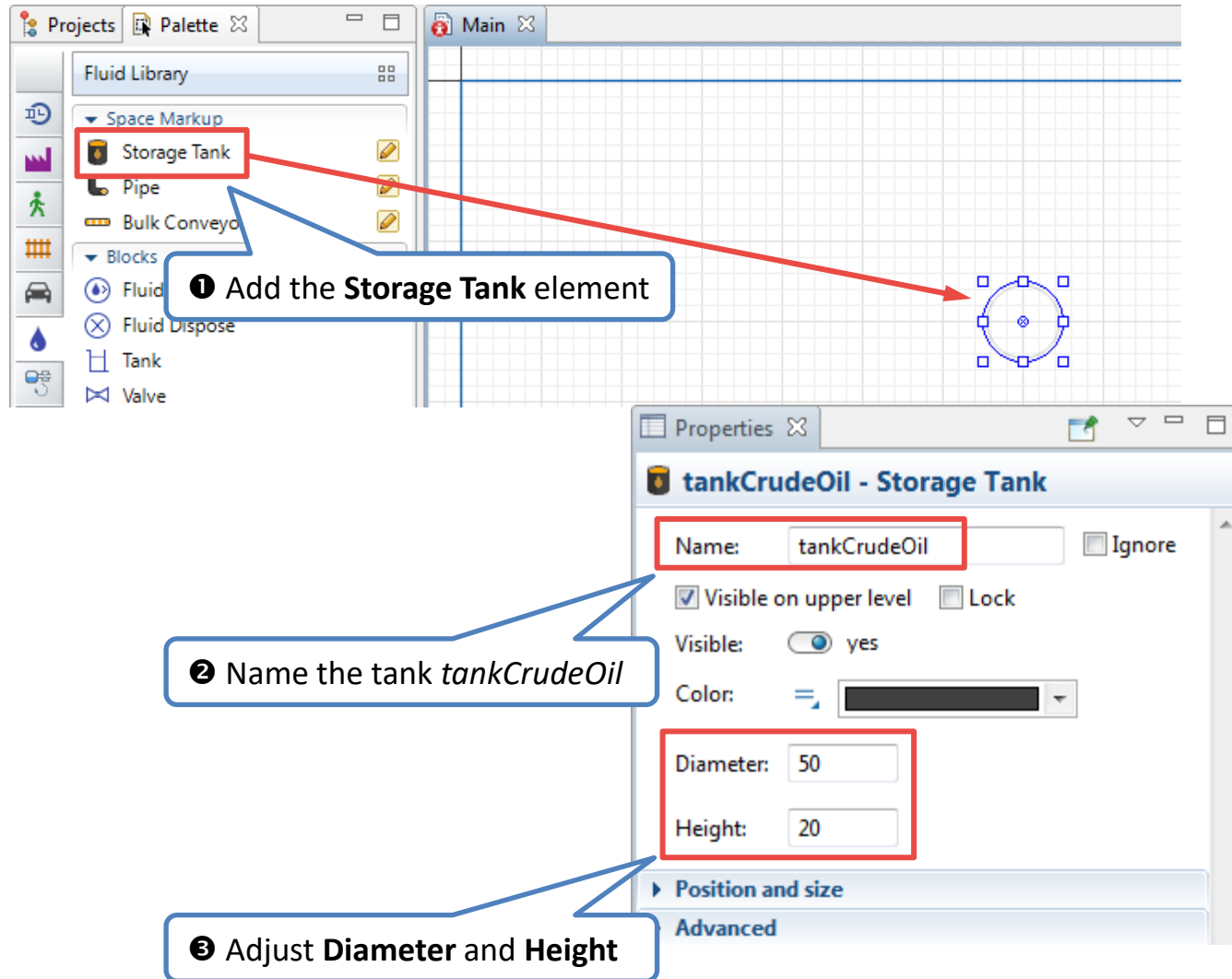
Oil Refinery Plant. Phase 1. Step 2



We will start with creating the flowchart to define the fluid movement.

1-6 Add the flowchart blocks from the **Fluid Library** palette that define crude oil entering the system, being transferred from the storage tank to the distillation tower via the pipe and leaving the system afterwards.

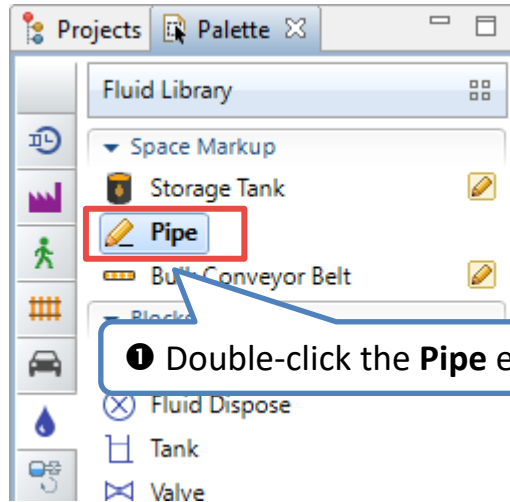
Oil Refinery Plant. Phase 1. Step 3



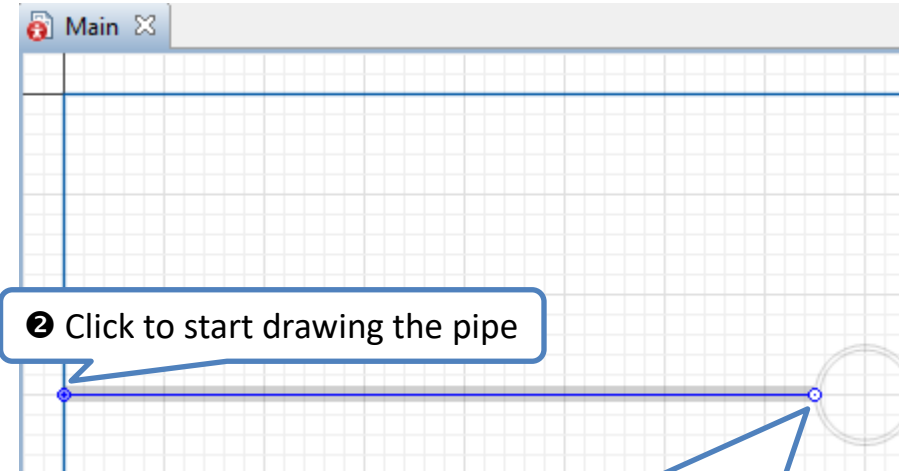
We will add the animation shape for the crude oil storage tank.

- 1 Drag the **Storage Tank** element from the **Fluid Library** palette onto the **Main** agent diagram.
- 2 Name the tank *tankCrudeOil*.
- 3 Set **Diameter** to 50 and **Height** to 20.

Oil Refinery Plant. Phase 1. Step 4



❶ Double-click the **Pipe** element



❷ Click to start drawing the pipe

❸ Double-click to finish drawing



❹ Draw one more pipe

Now we will add animation shapes for the pipes.

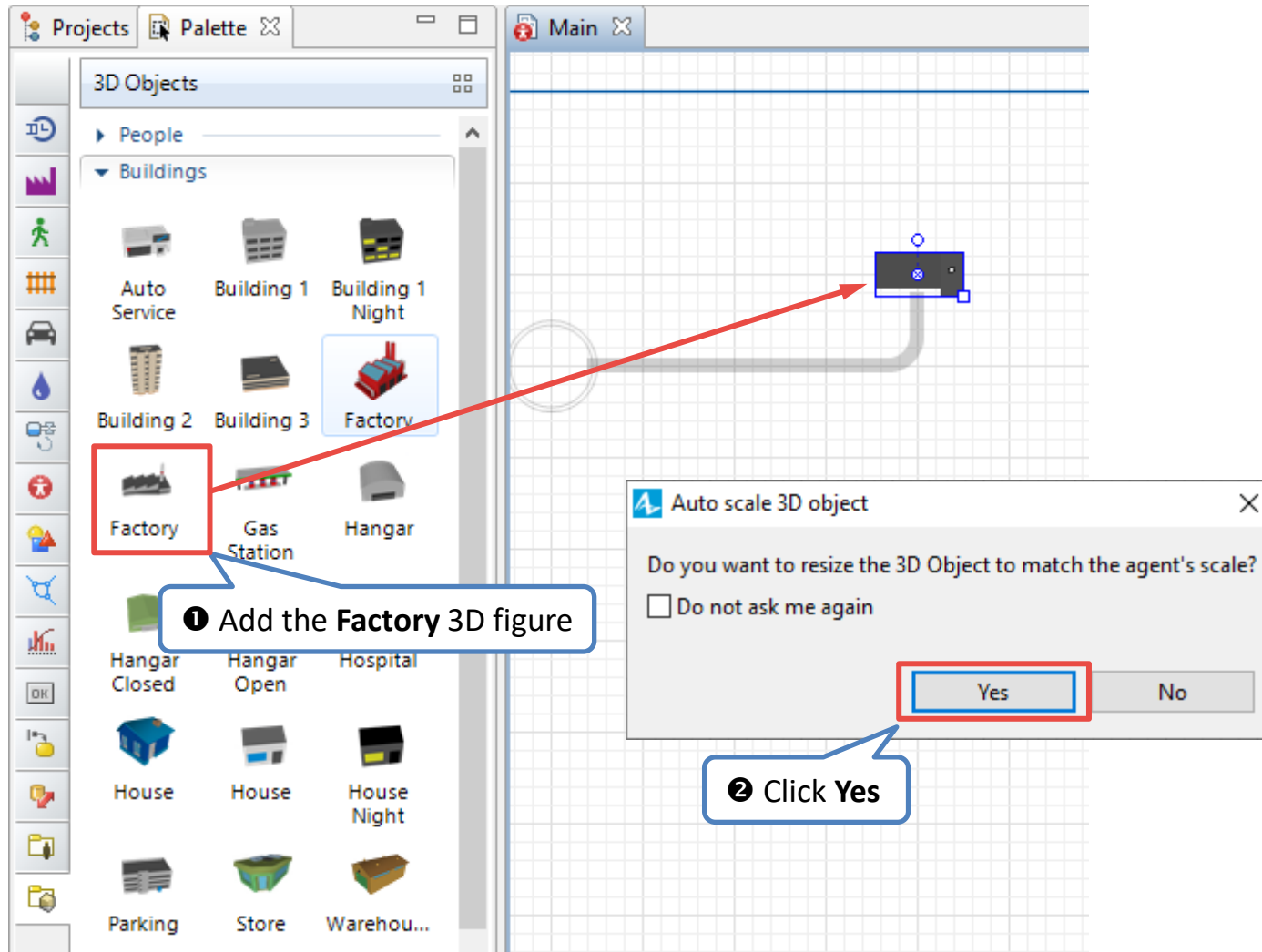
- ❶ Double-click the **Pipe** element in the **Fluid Library** palette to enter drawing mode.
- ❷-❸ Click in the pipe's starting point to start drawing. Draw the pipe leading into the crude oil storage tank. Double-click in the pipe's end point to finish drawing the pipe.
- ❹ Draw the pipe coming out of the crude oil storage tank. Note that this pipe has a bend. To add a bend when drawing a pipe, click in the bend point.

Pipe direction

Pipes are unidirectional. The liquid will always flow from the pipe's start point to its end point. To view the pipe direction, select the pipe. The start point is blue-colored; the end point is white-colored:



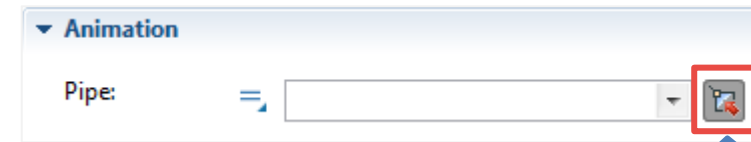
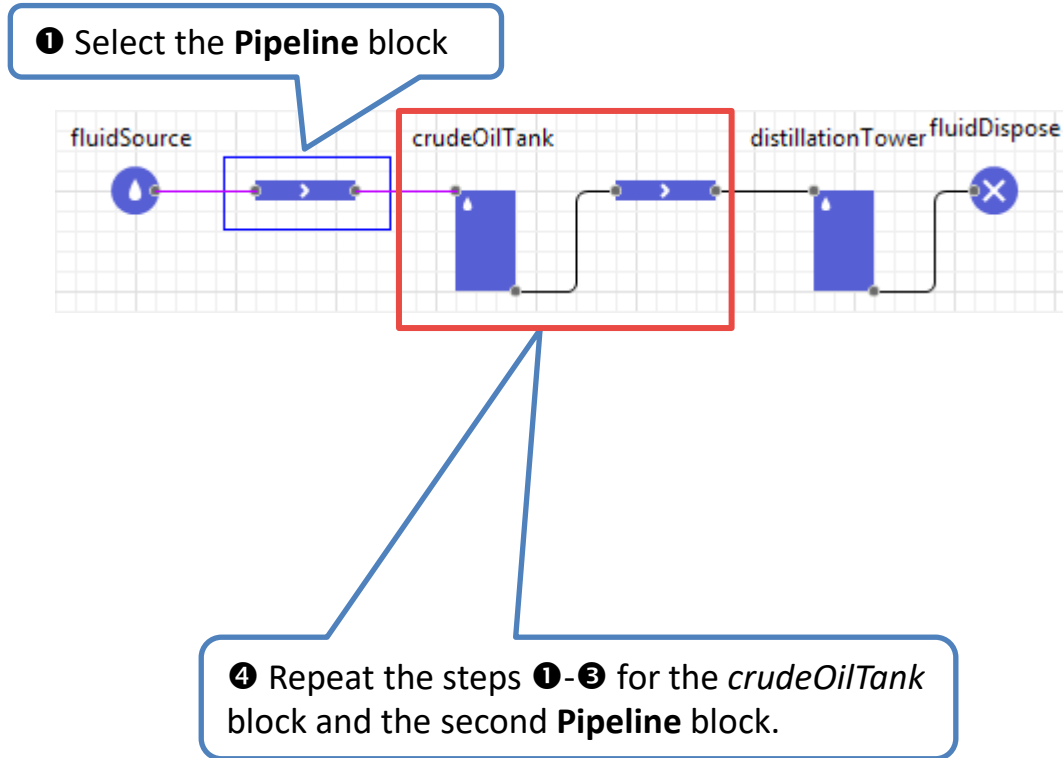
Oil Refinery Plant. Phase 1. Step 5



We will add a 3D object to represent the distillation tower.

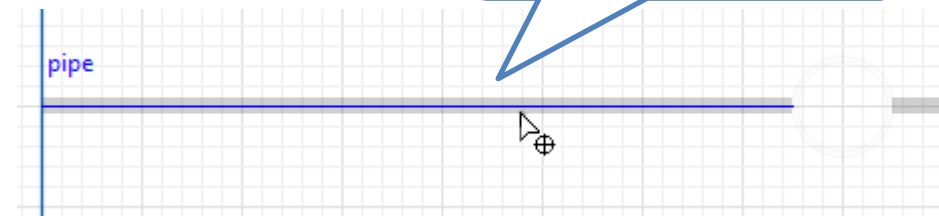
- 1 Drag the **Factory** element from the **Buildings** section of the **3D Objects** palette onto the **Main** agent diagram. Position it so that it appears visually connected to the crude oil storage tank via the pipe.
- 2 AnyLogic will prompt you to automatically adjust the scale of the 3D object to match the agent scale. Click **Yes**.

Oil Refinery Plant. Phase 1. Step 7



❷ Click this button next to the **Pipe** list

❸ Click the **Pipe** to link it to the flowchart block.



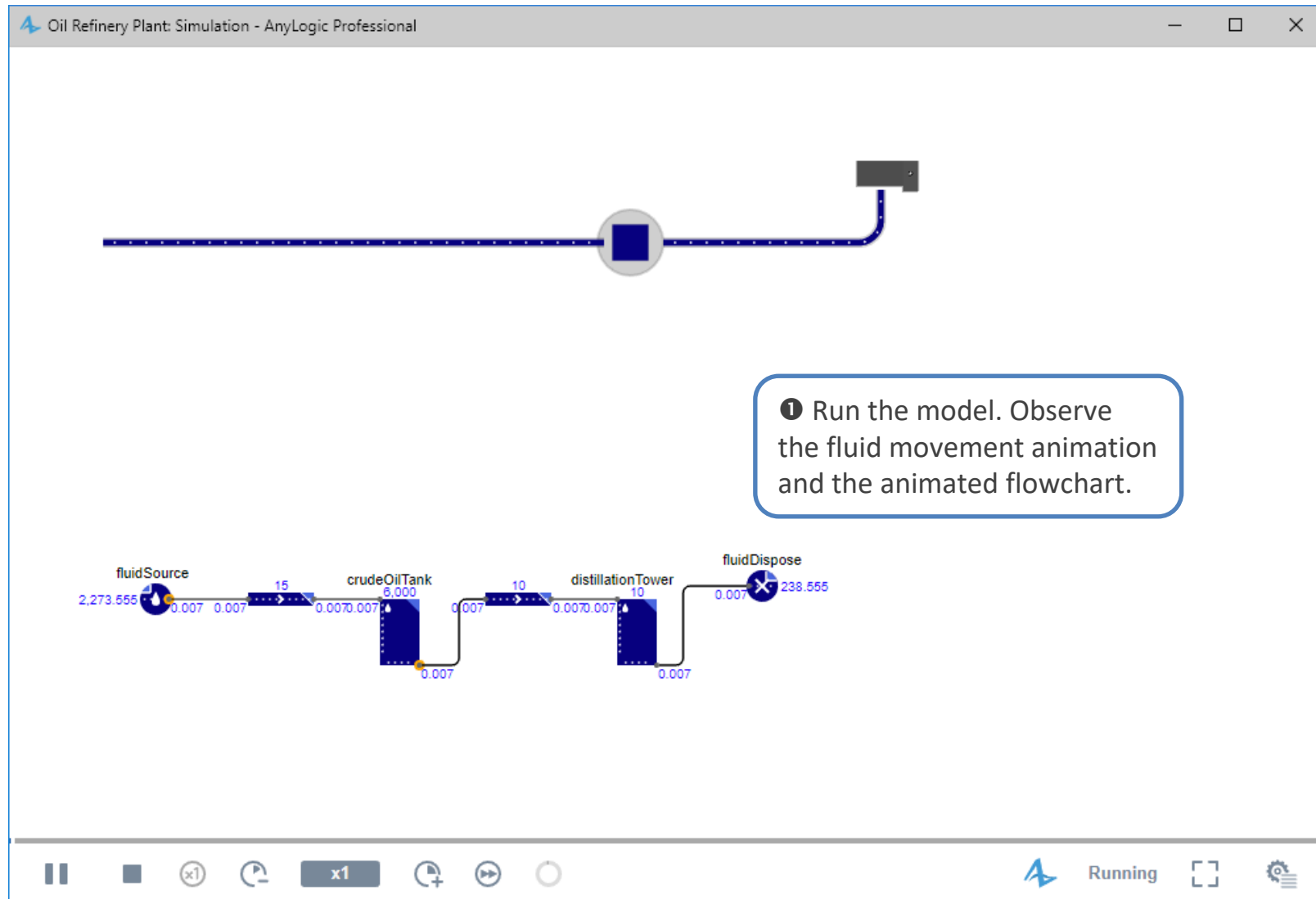
To animate the fluid flow, we need to link the flowchart blocks to the markup elements.

- ❶ Select the first **Pipeline** flowchart block and navigate to its properties.
- ❷ Expand the **Animation** section and click the  button next to the **Pipe** list. The available for selection markup shapes (pipes) will be highlighted in the graphical editor.
- ❸ Click the first pipe to select it and thus link it to the first **Pipeline** flowchart block.
- ❹ Repeat the steps ❶-❸ to link the *tankCrudeOil* markup shape to the *crudeOilTank* flowchart block and the second pipe – to the second **Pipeline** flowchart block.

Animating the flowchart

Linking flowchart blocks to markup elements is only needed for animating the corresponding parts of the process. It does not affect the logic. Since we chose not to animate the *distillationTower* tank block but rather use a static 3D object to represent it, we do not need to link it to a markup shape.

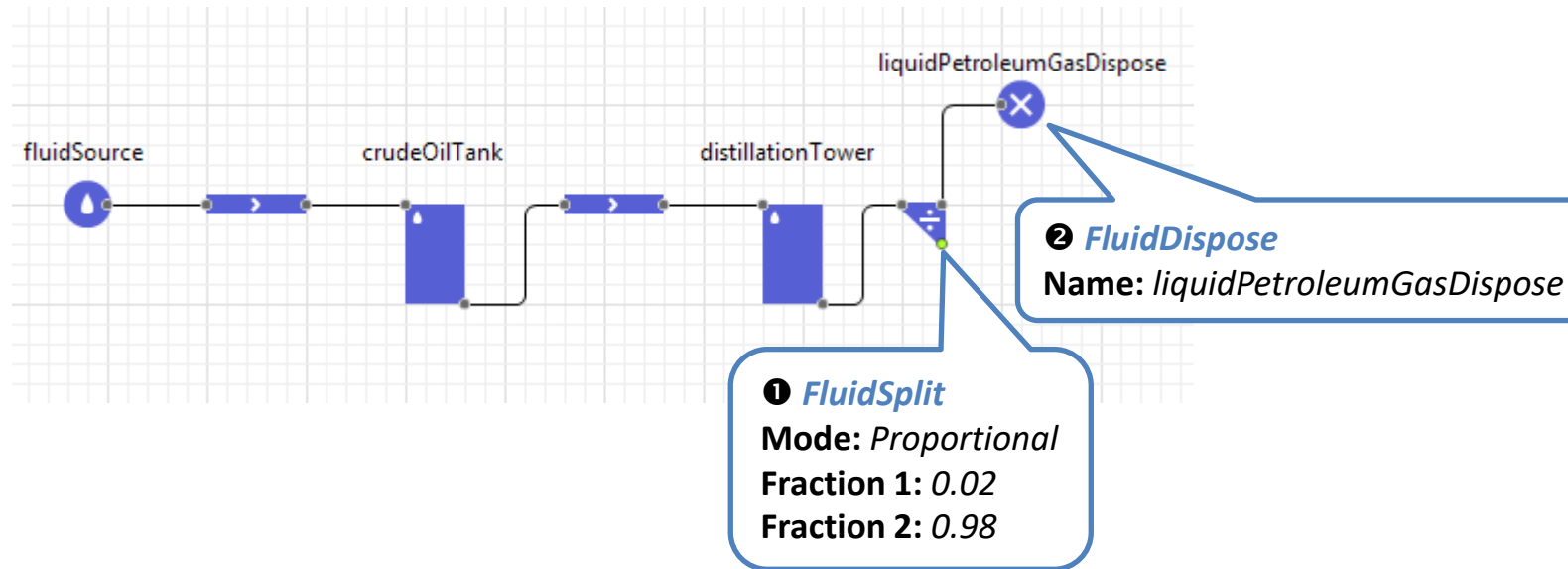
Oil Refinery Plant. Phase 1. Step 8



Oil Refinery Plant. Phase 2.

- In this phase, we will expand the model to include the process of fractional distillation. Four petroleum products will be produced as a result: liquid petroleum gas, gasoline, fuel oil and diesel oil.

Oil Refinery Plant. Phase 2. Step 1



We will start by adding two blocks to the flowchart, which will represent the production of liquid petroleum gas.

- 1 Place the **FluidSplit** block between the *distillationTower* and *fluidDispose* blocks.

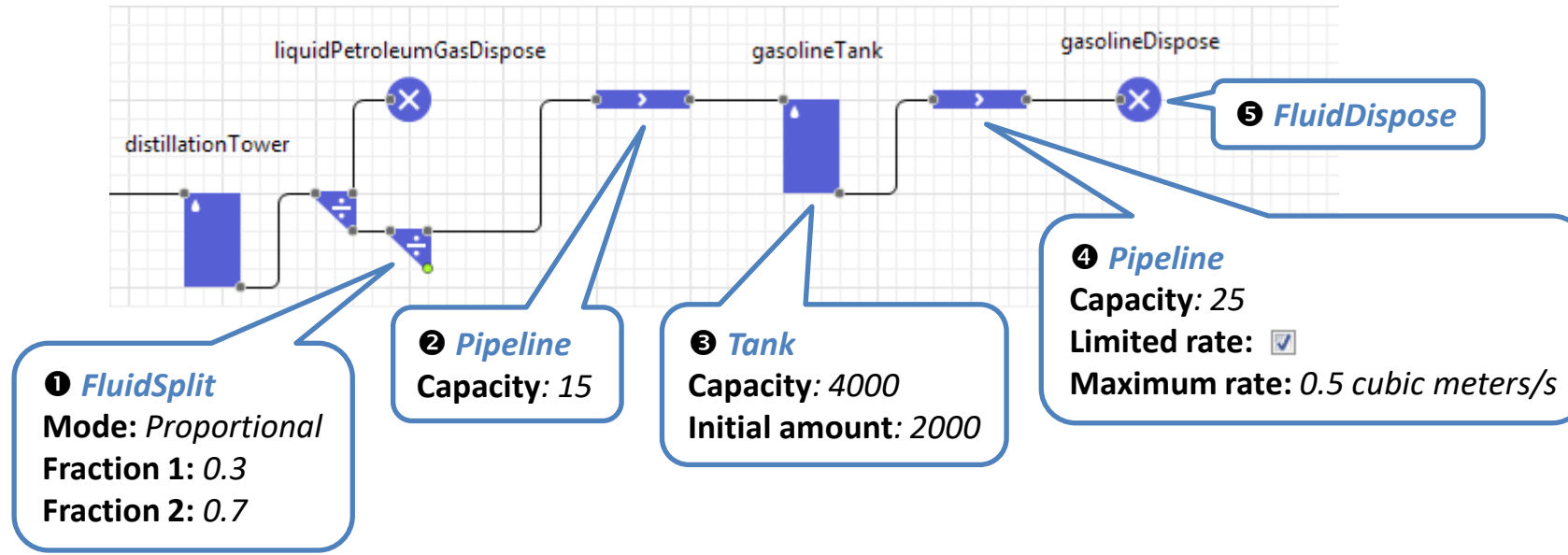
FluidSplit block

This block splits the input stream into two different output streams; the sum of the output flow rates equals the input flow rate.

The block can work in three alternative modes:

- *Neutral*, under which neither of the outputs gets priority.
- *Proportional*, under which the following constraint applies:
$$\text{output 1 rate} / \text{Fraction 1} = \text{output 2 rate} / \text{Fraction 2}$$
- *Priority*, under which the block tries to maximize the rate of one of the outputs, and the other output gets the remaining.

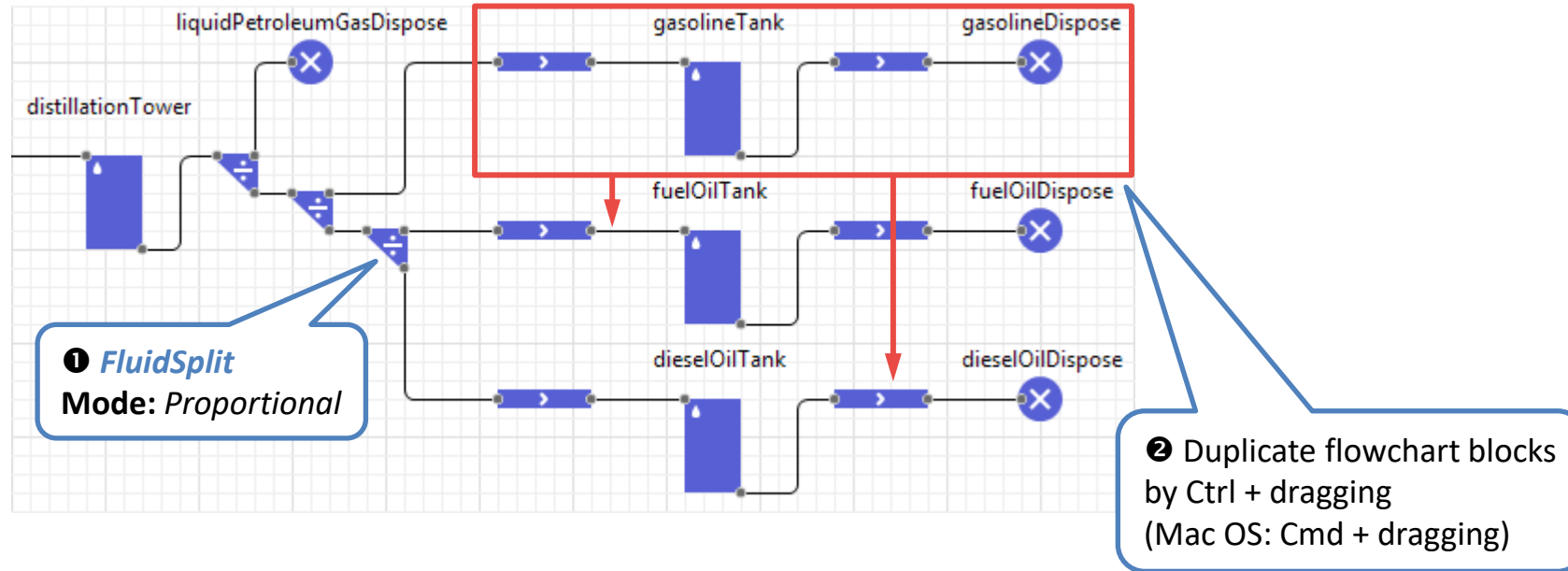
Oil Refinery Plant. Phase 2. Step 2



Let us expand the flowchart to model the production of gasoline.

①-⑤ Insert the **Fluid Library** blocks in the following sequence into the flowchart:
FluidSplit – Pipeline – Tank – Pipeline – FluidDispose

Oil Refinery Plant. Phase 2. Step 3



In the similar manner, we will add the flowchart blocks to model the processes of fuel oil and diesel oil production.

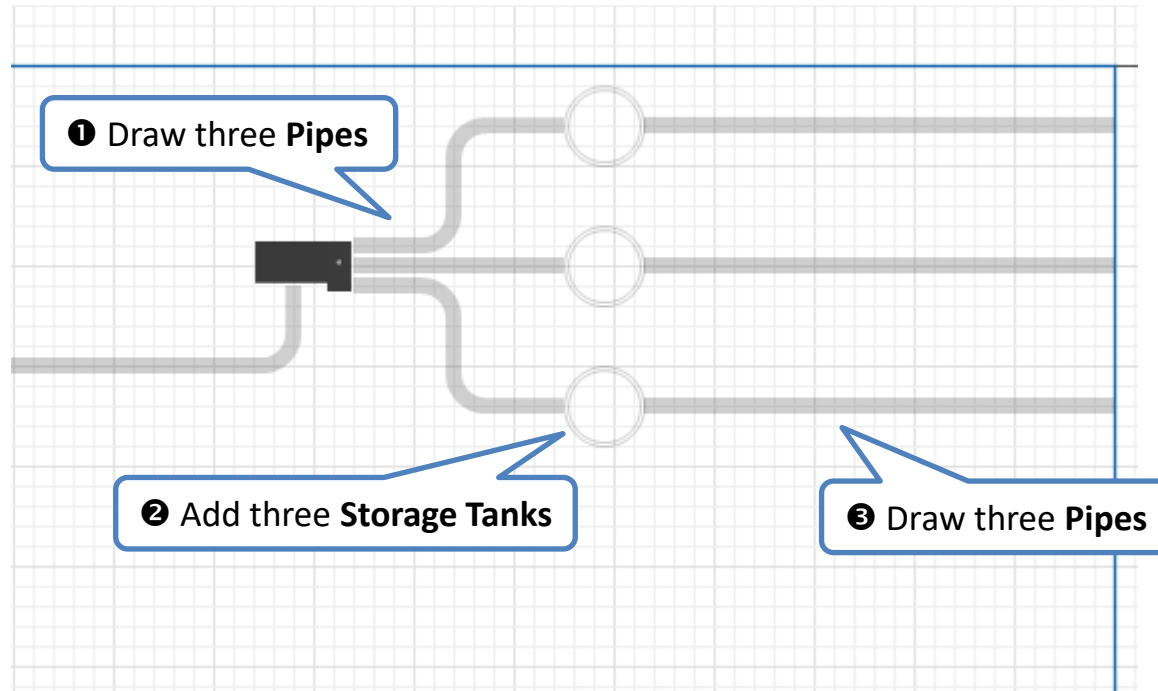
- ① Add the third **FluidSplit** block.
- ② Use Ctrl + dragging (Mac OS: Cmd + dragging) to create two copies of the sets of flowchart blocks for two processes: fuel oil production and diesel oil production. Rename the blocks for each process as displayed on the screenshot and connect them to the outputs of the **FluidSplit** block.

Why do we use three FluidDispose blocks?

In the **Process Modeling Library** you can connect the output ports of two blocks to one input port of another block.

In the **Fluid Library**, one output port can be connected to one input port only; otherwise, it is not clear how to split flows from different parts of the network. To model one-to-many connections, use the **FluidMerge**, **FluidSplit**, **FluidSelectInput** and **FluidSelectOutput** blocks.

Oil Refinery Plant. Phase 2. Step 4

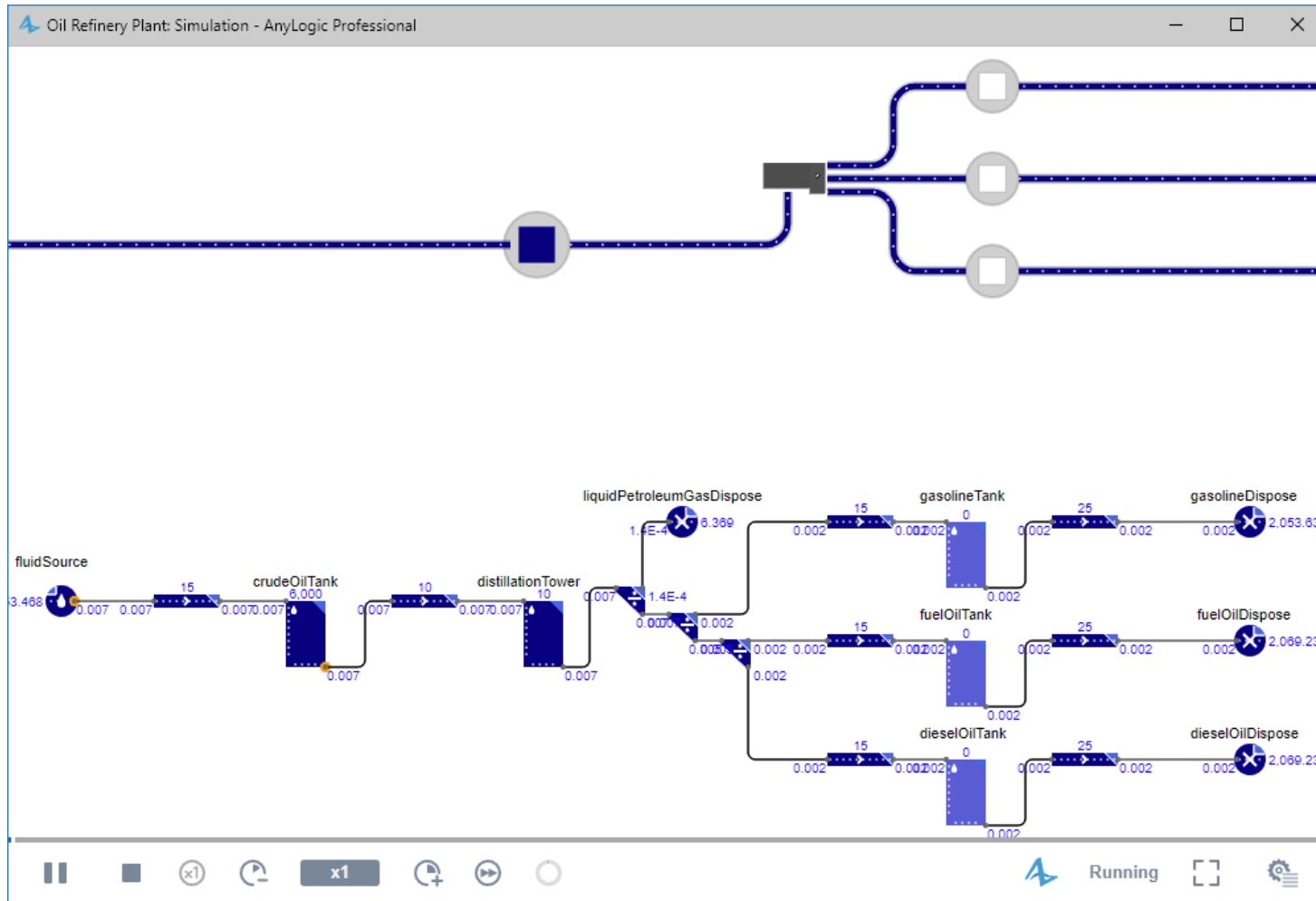


4 Link the flowchart blocks to the markup elements

Let us add markup shapes to animate the production sub-processes we have just defined via the flowchart. The resulting graphical diagram should resemble the one on the screenshot.

- ❶ Draw three pipes coming out of the **Factory** object.
- ❷ Add three storage tanks. Name the tanks *tankGasoline*, *tankFuelOil* and *tankDieselOil*.
- ❸ Draw a pipe coming out of each storage tank.
- ❹ We can now animate each of the three fuel production sub-processes (defined via the **Pipeline – Tank – Pipeline** sequence of flowchart blocks) by linking each block to the corresponding markup shape. In the **Animation** section of each block's properties, click the  button and select the appropriate markup shape in the graphical editor.

Oil Refinery Plant. Phase 2. Step 5



Run the model.

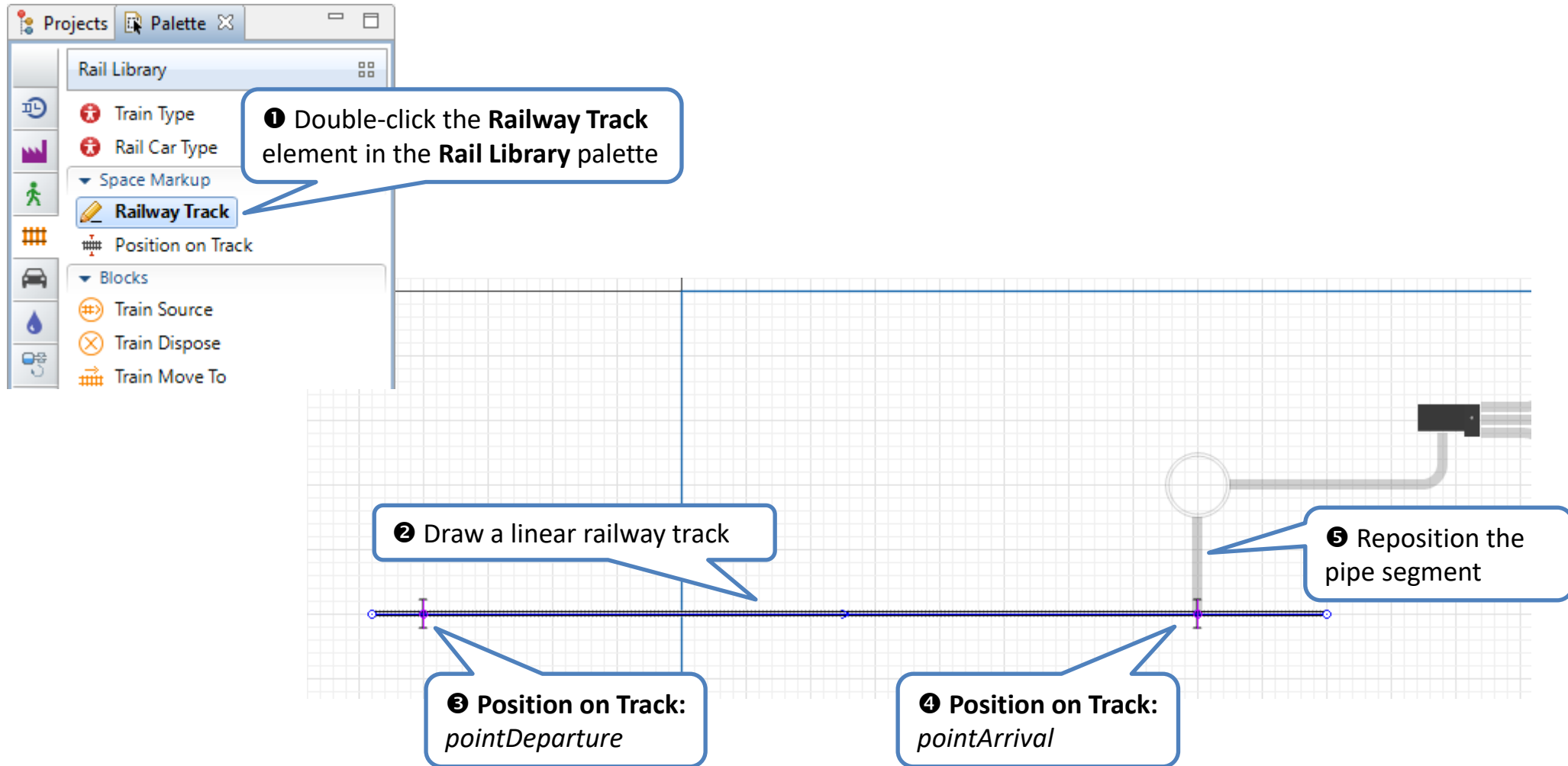
You may notice that the gasoline, fuel oil and diesel oil storage tanks get empty after a certain period of time. This is due to the nature of the **Fluid Library**, which dynamically adjusts the flow rates of the elements (namely, it decreases the **FluidSource** rate and increases the **FluidDispose** rates) to maximize the system's throughput.

In the next phase, we will gain more control over the fluid input and output by including shipments of crude oil and dispatches of refined products.

Oil Refinery Plant. Phase 3.

- In this phase, we will use the **Rail Library** blocks to include crude oil shipments by train in our model. We will also model dispatches of oil refinery products.

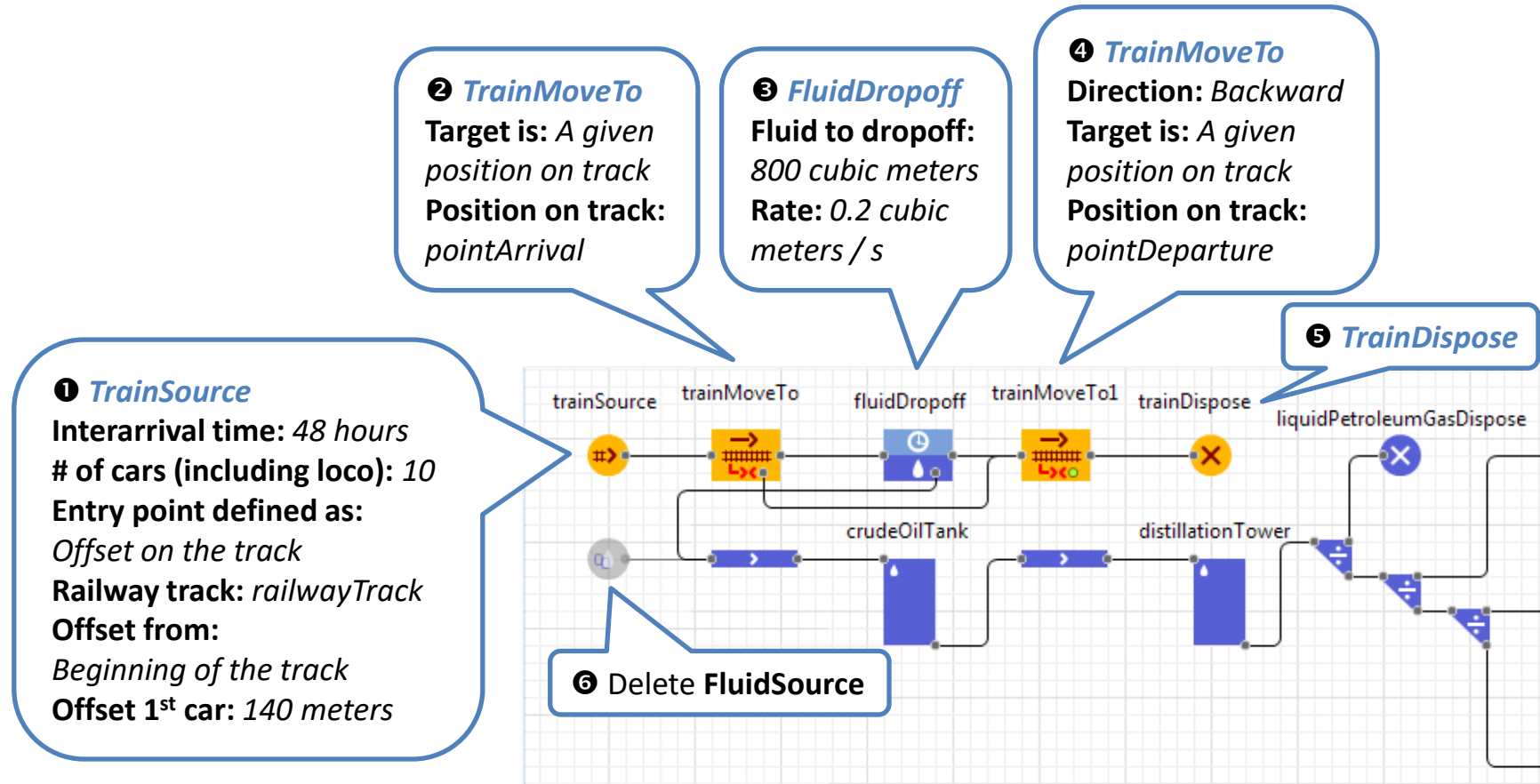
Oil Refinery Plant. Phase 3. Step 1



We will start by adding a railway track.

- ①-② Double-click the **Railway Track** element in the **Rail Library** palette to enter the drawing mode. Draw a linear railway track in the left to right direction starting from the outside of the visible area.
- ③-④ Drag two **Position on Track** elements from the **Rail Library** palette and place them onto the railway track as displayed on the screenshot. Name these elements *pointArrival* and *pointDeparture*.
- ⑤ Reposition the first segment of the pipe to visually connect the crude oil storage tank with the *pointArrival* position on track.

Oil Refinery Plant. Phase 3. Step 2

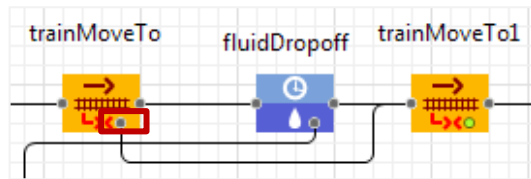


We will add the sub-process of delivering crude oil by train and use it instead of the **FluidSource** flowchart block.

①-⑤ Add the following blocks to the flowchart:

TrainSource – TrainMoveTo – FluidDropoff – TrainMoveTo – TrainDispose

Note that we connect **two** output ports of the *trainMoveTo* block with the *trainMoveTo1* block to prevent trains collisions:



If the arrival point is already occupied by a train, a new train will not move to it but will rather move back and disappear from the system.

⑥ Delete the **FluidSource** block and its connection with the **Pipeline** block.

FluidDropoff block

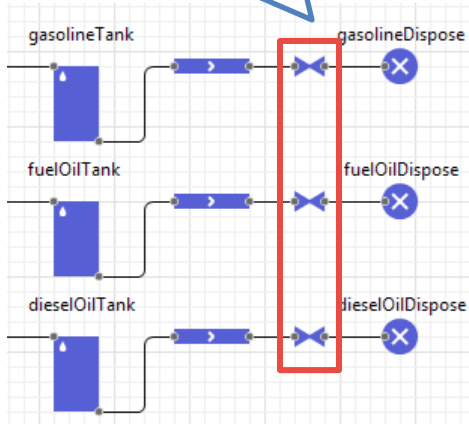
The **FluidDropoff** block of the **Fluid Library** drops off fluid from the agents passing through. An agent, which is a train in our case, is assumed to have a certain amount of fluid to drop off. Upon arrival of an agent, the block starts the outflow. By adding this block and connecting its **outFluid** port to the first **Pipeline** block we are effectively replacing the **FluidSource** block, which is no more needed.

Oil Refinery Plant. Phase 3. Step 3

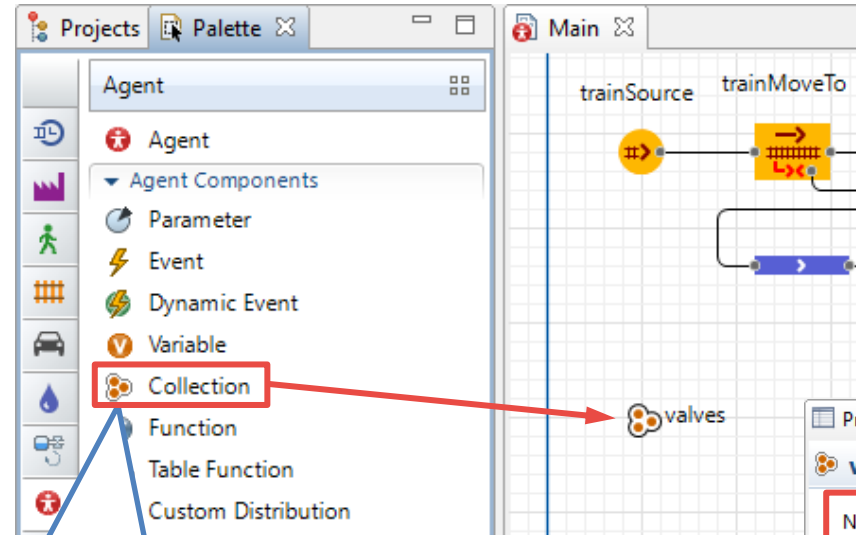
1 Valve

Maximum rate when open: ☒

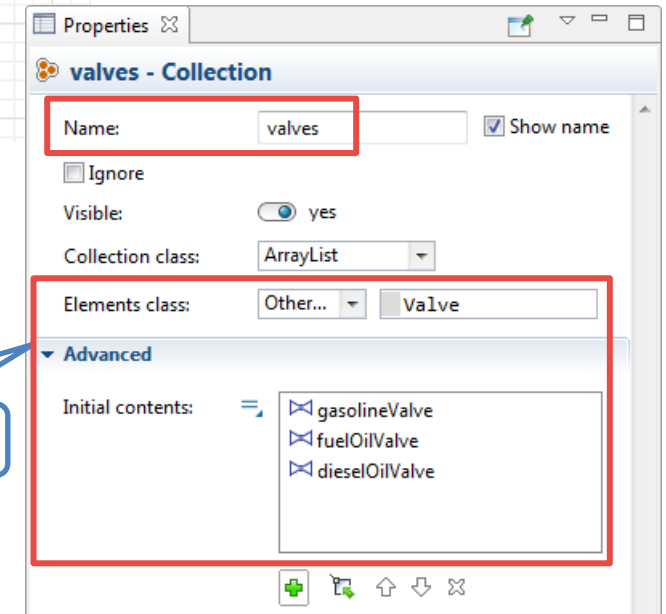
Initially closed: ☒



2 Add a Collection element



3 Set up the collection



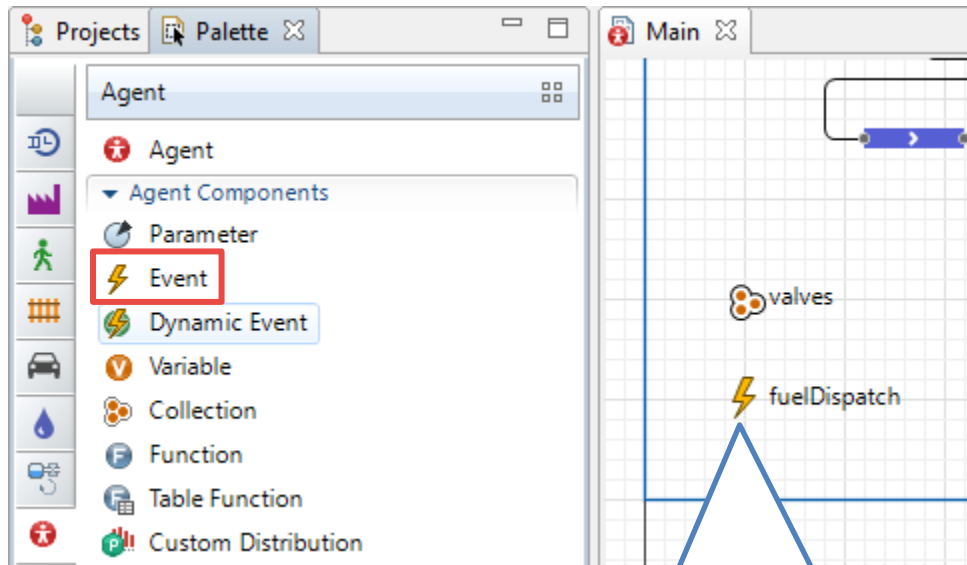
Crude oil in our model is now delivered by train. To model dispatches of refined products, we will block the flow by adding valves before the **FluidDispose** blocks and implement the logic for periodic valves opening.

- ➊ Add three **Valve** blocks from the **Fluid Library** palette, one for each oil product. Name the valves *dieselOilValve*, *fuelOilValve* and *gasolineValve* and enable the **Initially closed** option for each of them. Ensure that each valve is properly connected to the corresponding **Pipeline** and **FluidDispose** blocks.

Let us aggregate valves into a collection: this will provide us with an easier access to them in our model.

- ➋ Drag the **Collection** element from the **Agent** palette onto the **Main** diagram.
- ➌ Name the collection *valves* and set it up as displayed on the screenshot. After setting the collection **Elements class** to *Other > Valve*, click the  button below the **Initial contents** section. The **Valve** flowchart blocks will be available for selection and you can click each of them to add it to the collection.

Oil Refinery Plant. Phase 3. Step 4



① Event

Name: *fuelDispatch*

Trigger type: *Timeout*

Mode: *Cyclic*

Recurrence time: *8 hours*

Action: *randomFrom(valves).dispense(140);*

Since the valves are initially closed, we need to add logic for their periodic opening, which will simulate the dispatch of a certain product.

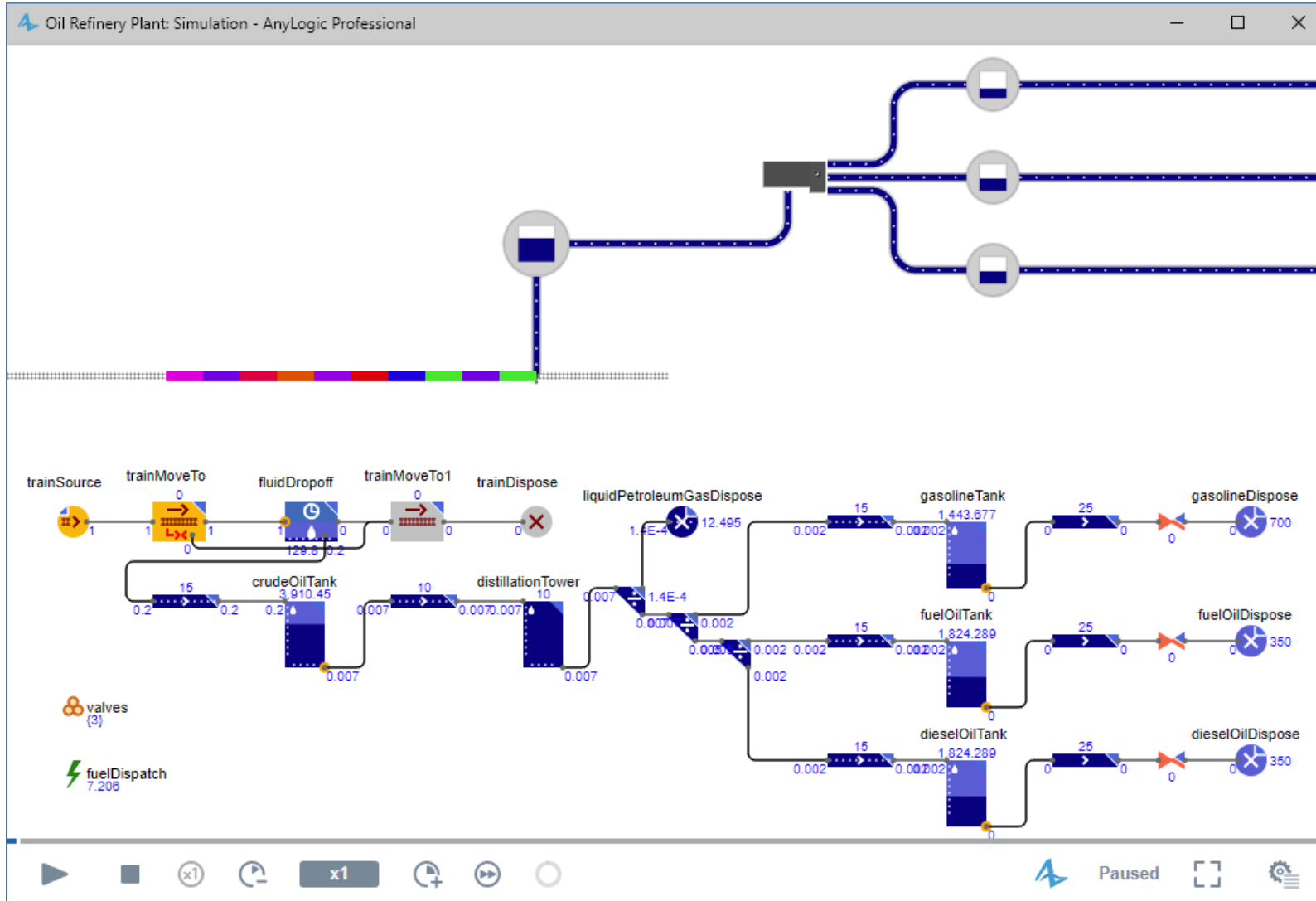
① Drag an **Event** element from the **Agent** palette onto the **Main** diagram.

② Name the event *fuelDispatch*. In the **Properties** section, set the event to be triggered three times a day. Provide the following code as the event's **Action**:

```
randomFrom(valves).dispense(140);
```

We call *randomFrom(valves)* to obtain a random **Valve** block from the *valves* collection. The *dispense()* function of the valve opens it and dispenses the specified fluid amount, 140 cubic meters in our case. The valve will automatically close as soon as the fluid amount is dispensed.

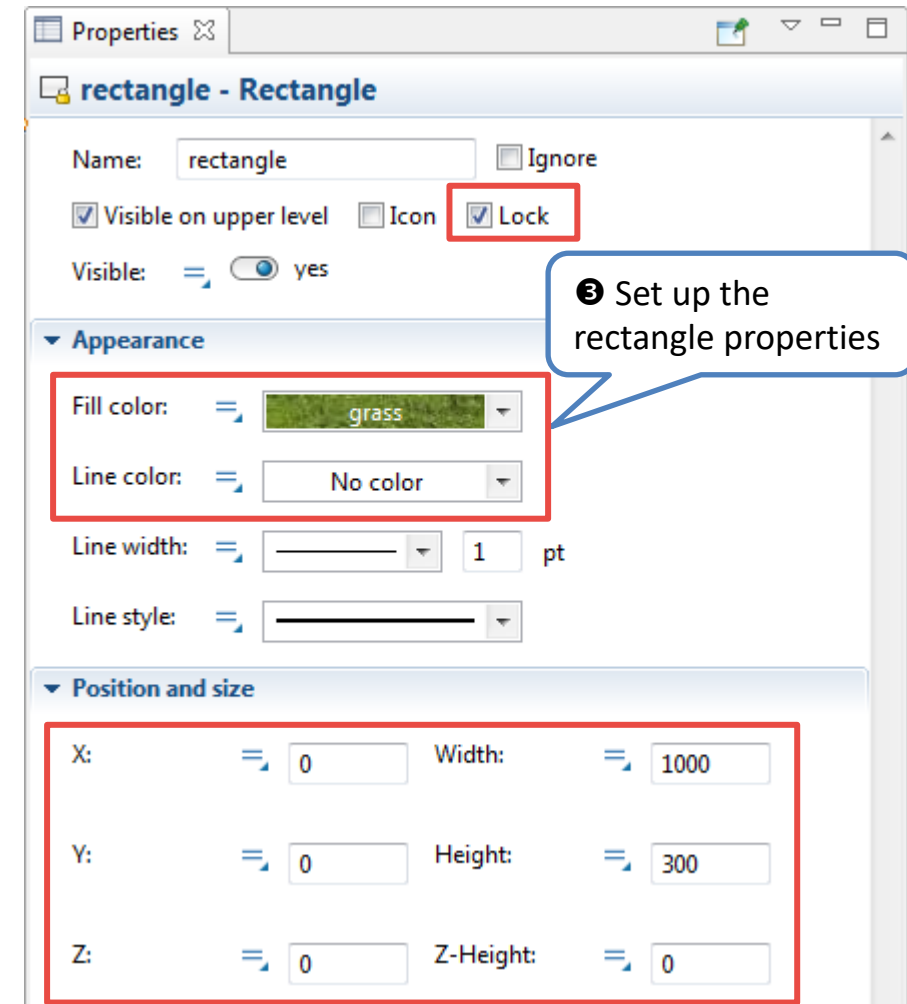
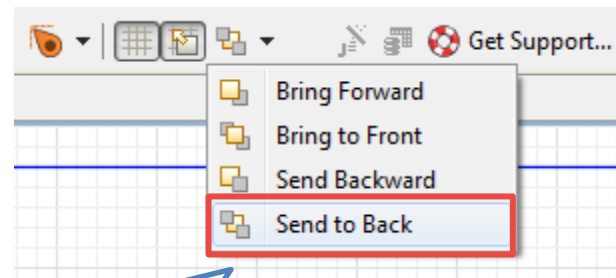
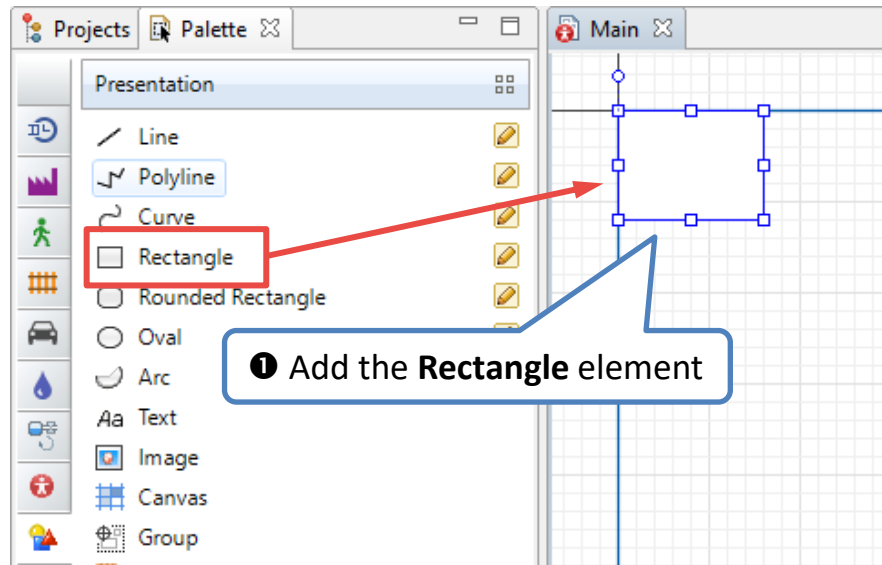
Oil Refinery Plant. Phase 3. Step 5



Oil Refinery Plant. Phase 4.

- As you can see, the model behavior is now more realistic, with crude oil being shipped by train and oil refinery products being periodically dispatched.
- In this phase, we will improve the model appearance and add 3D animation to it.

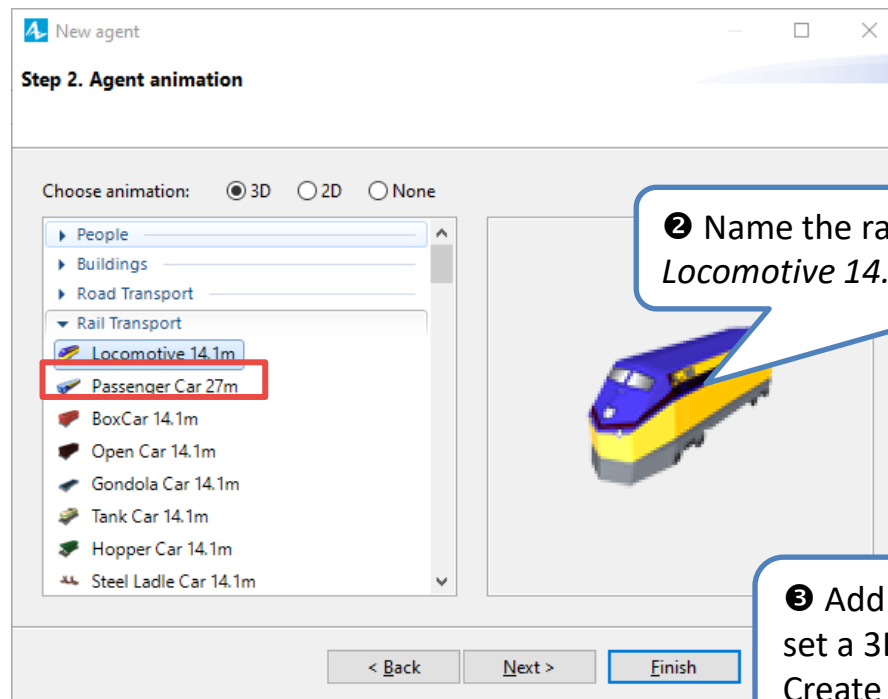
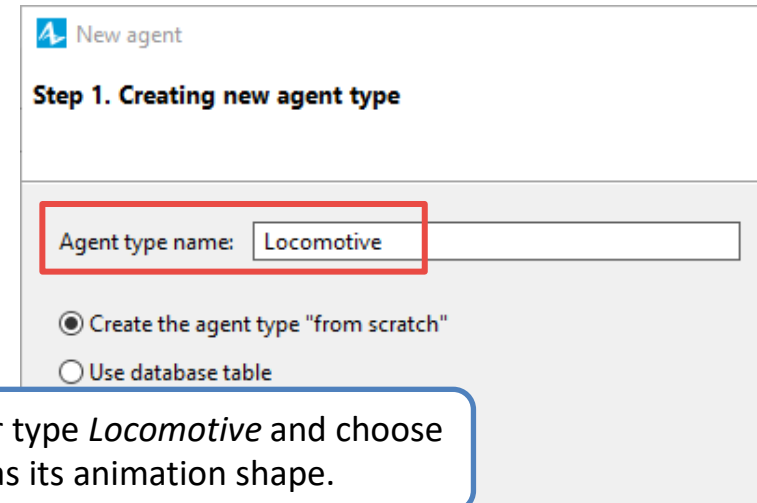
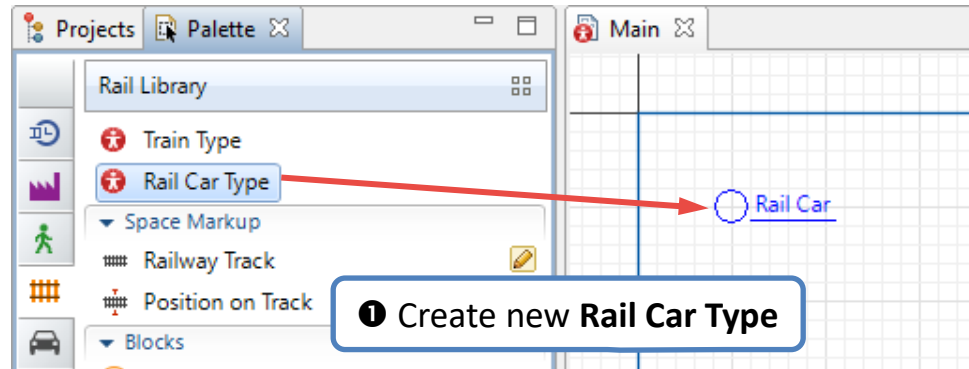
Oil Refinery Plant. Phase 4. Step 1



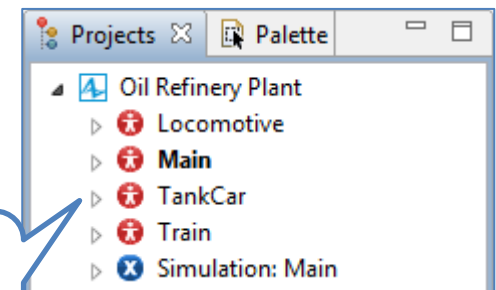
Now we will enhance the model animation layout.

- ❶ Add the **Rectangle** element from the **Presentation** palette onto the **Main** agent diagram.
- ❷ Send the rectangle to back to prevent it from overlaying the markup shapes.
- ❸ Set the **Rectangle** element's properties:
 - Select the **Lock** check box. Since we will not need to move or resize the rectangle, we can lock it in the graphical editor. This will prevent us from accidentally selecting it.
 - Set the **Fill color** option to **Textures...** and select the *Grass* texture in the dialog box that opens. Set the **Line color** option to *No color*.
 - Adjust the rectangle **Position and size** properties. Set **X** and **Y** to *0* to match the rectangle top-left corner with the axis origin. Set **Width** and **Height** to *1000* and *300* to stretch the rectangle across the upper half of the visible area marked by the blue rectangle. Set **Z-Height** to *0* so that the rectangle does not cover the objects placed on top of it when viewing 3D animation.

Oil Refinery Plant. Phase 4. Step 2



3 Add another Rail Car type: *TankCar* and set a 3D shape for it. Create a new Train Type: *Train*.



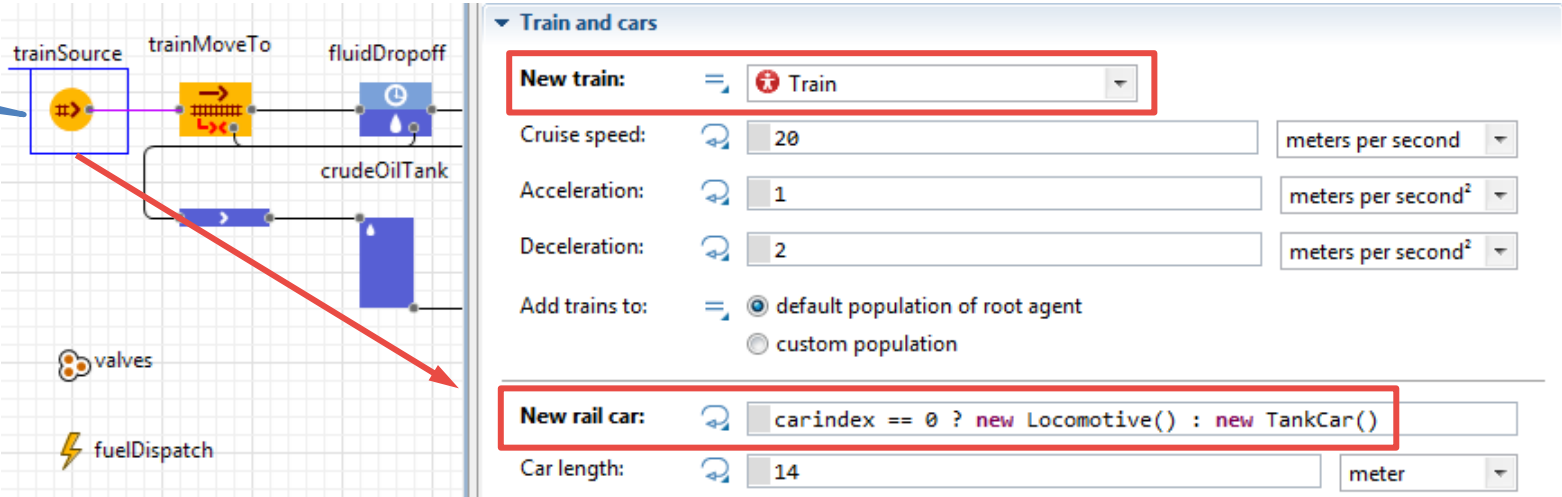
By now, the train is represented by a set of colored rectangles.

We will customize its appearance by creating two rail car types: *Locomotive* and *TankCar*. When creating new rail car types, choose the corresponding 3D animation objects from the list.

Similarly, create a **Train Type** by dragging **Train Type** from the **Rail Library** palette. Name it *Train*. You do not need to choose any animation figure for the train: it will consist of the cars.

Oil Refinery Plant. Phase 4. Step 3

❶ Adjust the properties of the *trainSource* block



The screenshot shows the 'Train and cars' properties panel for the 'trainSource' block. The 'New train' dropdown is set to 'Train'. The 'New rail car' text field contains the code `carindex == 0 ? new Locomotive() : new TankCar()`. Other properties include 'Cruise speed' (20 m/s), 'Acceleration' (1 m/s²), 'Deceleration' (2 m/s²), and 'Car length' (14 meter).

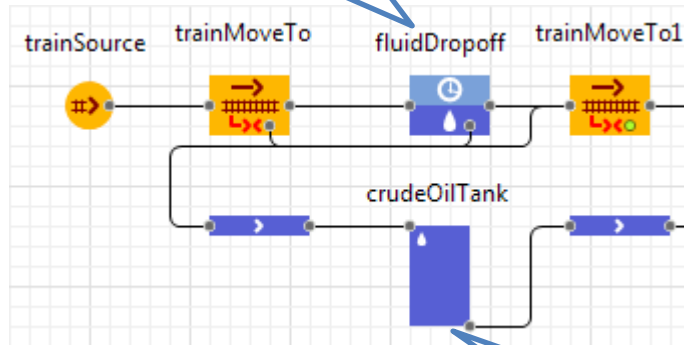
Now we will set the *trainSource* block to generate trains consisting of the rail cars that we created earlier.

- ❶ In the **Trains and cars** section of the *trainSource* block's properties, set **New train** to *Train*. Provide the following code in the **New rail car** field:
`carindex == 0 ? new Locomotive() : new TankCar()`

This expression sets *Locomotive* as the first car of the train and *Tank Cars* – as the remaining cars.

Oil Refinery Plant. Phase 4. Step 4

- ① Custom batches: ☒
- Custom batch color: ☒
- Batch color: black



- ② Custom initial batch: ☒
- Custom batch color: ☒
- Batch color: black

Adjust the properties of the *fluidDropoff* and *crudeOilTank* blocks.

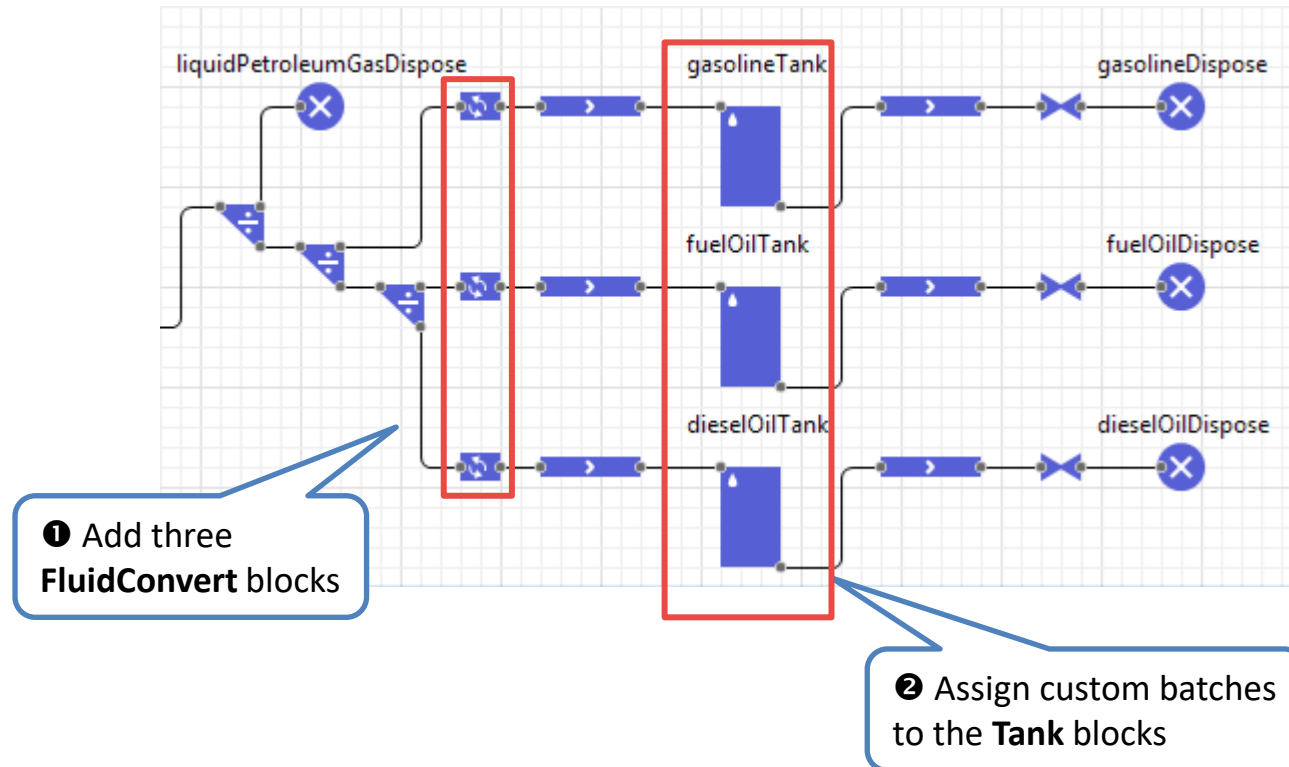
The fluid in our model is currently of the navy blue color. To assign custom colors to the fluid passing various blocks of the flowchart, we will use *batches*.

Configure the *fluidDropoff* and *crudeOilTank* blocks to use custom batches. Set the custom batch color to *black*.

Batches

Fluid or bulk matter in the Fluid Library can be of different types, called batches. Whenever a new batch is created, you can specify the color of the batch. That color will then show in the flowchart and in the animation. The batch-to-color mapping, however, is not maintained: if you, say, set yellow color for the “Diesel” batch in one object, and brown color for the batch also called “Diesel” in another object, these two batches will have different colors unless they merge, in which case one of the colors will be kept.

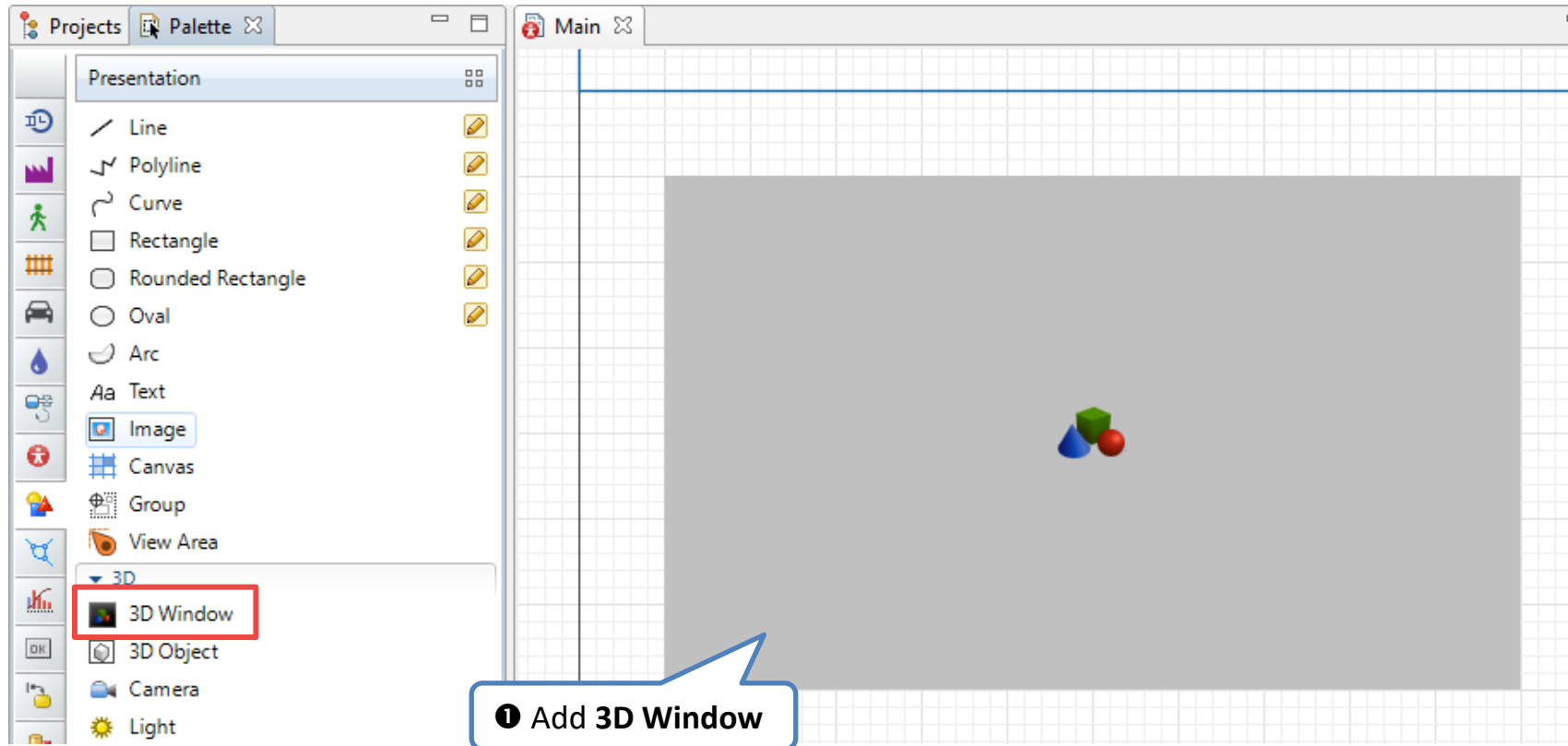
Oil Refinery Plant. Phase 4. Step 5



Now we will assign custom-colored batches to each refined product.

- 1 Drag three **FluidConvert** blocks from the **Fluid Library** palette, one for each refined product. Position each block between the **FluidSplit** block and the corresponding pipe segment. For each block, set the **Custom output batch** option, then select the **Custom batch color** option and finally set a different **Batch color**:
 - First block: **khaki**
 - Second block: **brown**
 - Third block: **darkGoldenRod**
- 2 Similarly, enable custom batches for each of the three **Tank** blocks and set the same batch colors for them.

Oil Refinery Plant. Phase 4. Step 6



Now we will add 3D animation to our model.

- 1 Drag the **3D Window** element from the **Presentation** palette onto the **Main** diagram and place it below the visible area.

Oil Refinery Plant. Phase 4. Step 7

The screenshot displays the AnyLogic Professional interface for the 'Oil Refinery Plant' simulation. The main window is split into two views: a 2D top-down view at the top and a 3D perspective view at the bottom. The 2D view shows a network of pipes and storage tanks with colored batches (yellow, red, orange) representing different oil products. The 3D view shows the same network in a 3D perspective, with tanks and pipes rendered in 3D. A callout box labeled '1 Run the model' points to the 'Run' button in the top-left corner. Another callout box labeled '2 Open the developer panel' points to the 'Developer' button in the bottom-right corner. A third callout box labeled '3 Switch between 2D and 3D animation. Notice the custom colored batches that are displayed both in 2D and 3D animation as well as the flowchart.' points to the 'window3d' button in the top-right corner. The interface also includes a status bar at the bottom showing 'Running' and '34% of 256M'.

1 Run the model

3 Switch between 2D and 3D animation. Notice the custom colored batches that are displayed both in 2D and 3D animation as well as the flowchart.

2 Open the developer panel

Oil Refinery Plant. Phase 5.

- In this phase, we will add statistics to our model to observe the system performance.

Oil Refinery Plant. Phase 5. Step 1

The screenshot shows the AnyLogic software interface. On the left, the 'Projects' tree displays the hierarchy: 'Oil Refinery Plant' > 'Main' > 'Simulation: Main'. A blue callout box with the text '1 Select Simulation' points to the 'Simulation: Main' item. On the right, the 'Properties' window for 'Simulation - Simulation Experiment' is open. It shows fields for 'Name' (Simulation), 'Top-level agent' (Main), and 'Maximum available memory' (256 Mb). Under the 'Model time' section, the 'Execution mode' is set to 'Real time with scale' (1). A red rectangle highlights the 'Stop' settings: 'Stop at specified time', 'Start time: 0', and 'Stop time: 2400'. A blue callout box with the text '2 Adjust Stop time' points to this red rectangle. Below the red rectangle, 'Start date' is set to 16.11.2016 and 'Stop date' is set to 24.01.2017.

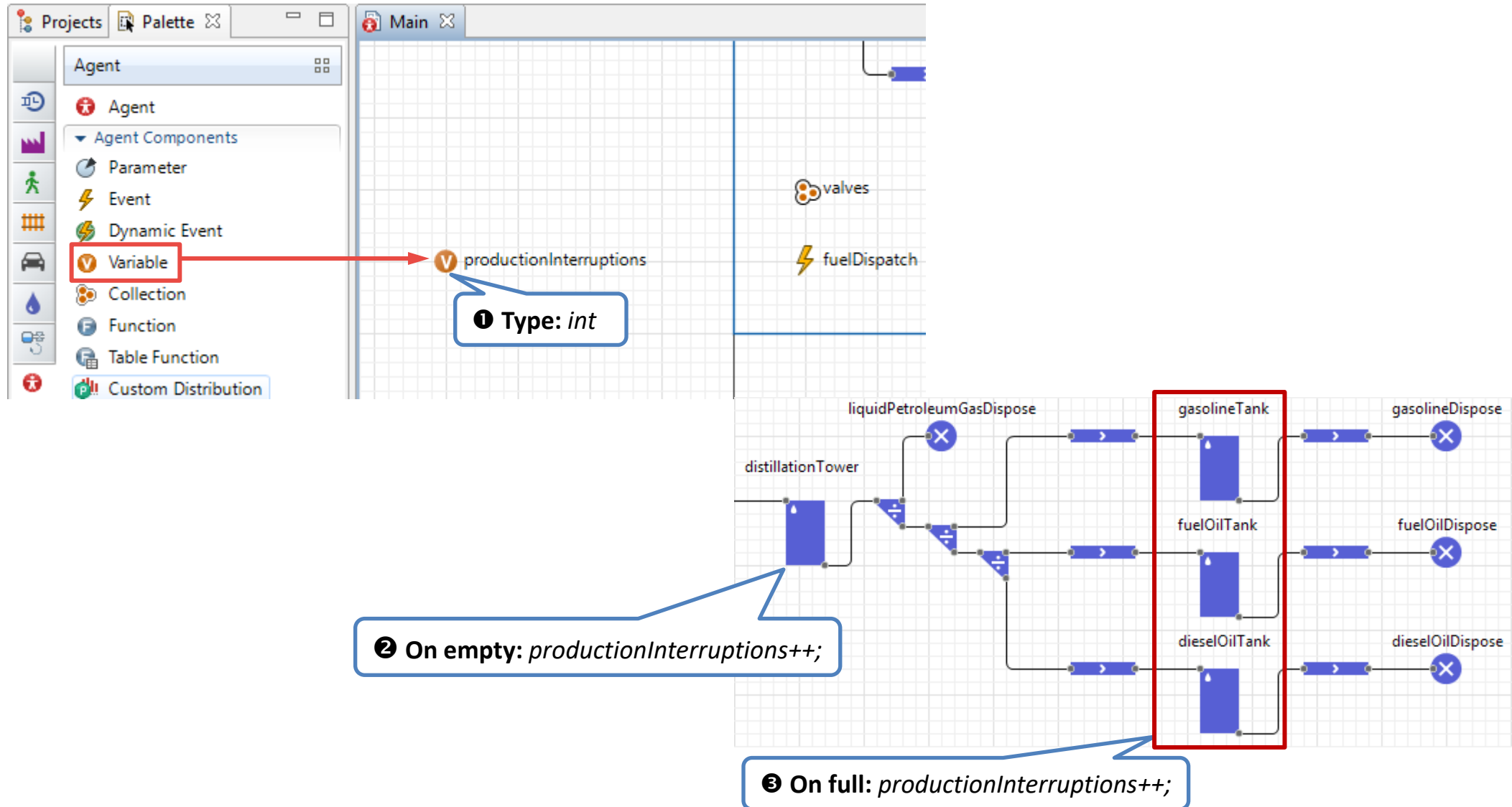
1 Select **Simulation**

2 Adjust **Stop time**

Let us limit the simulation time horizon to 100 days.

- 1 Select the **Simulation** experiment in the **Projects** tree.
- 2 In the **Model time** section of its properties, set the experiment to **Stop at specified time**.
Since our model's time units are hours, 100 days correspond to 2400 hours. Set **Stop time** to 2400.

Oil Refinery Plant. Phase 5. Step 2

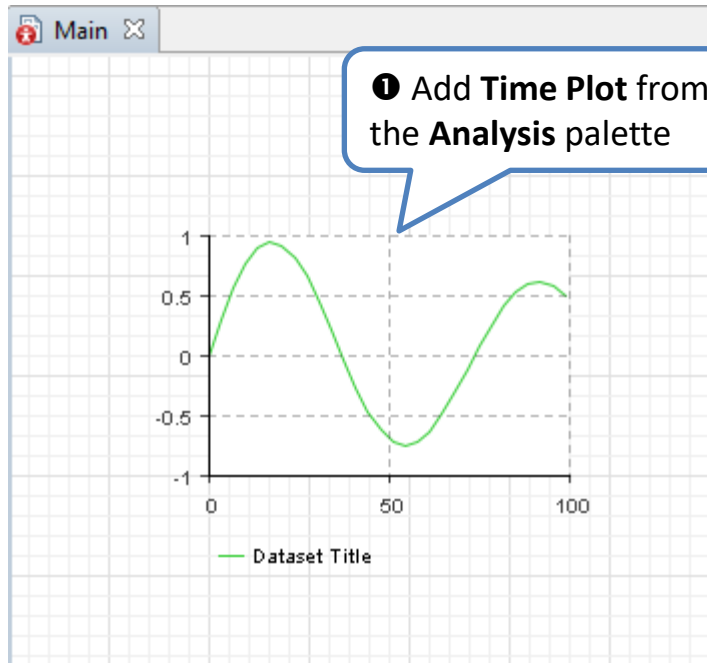


We will set up the calculation of production interruptions, which occur when either the distillation tower is empty or any of the refined product containers is full and cannot accept more product.

- ❶ Add the *productionInterruptions* variable and set its **Type** to *int*.
- ❷-❸ Provide the following code in the **On empty** action field of the *distillationTower* block as well as the **On full** action field of the *gasolineTank*, *fuelOilTank* and *dieselOilTank* blocks:
productionInterruptions++;

This action will increment the *productionInterruptions* variable when a production interruption occurs.

Oil Refinery Plant. Phase 5. Step 3



2 Add chart data items for crude oil and refined products tanks

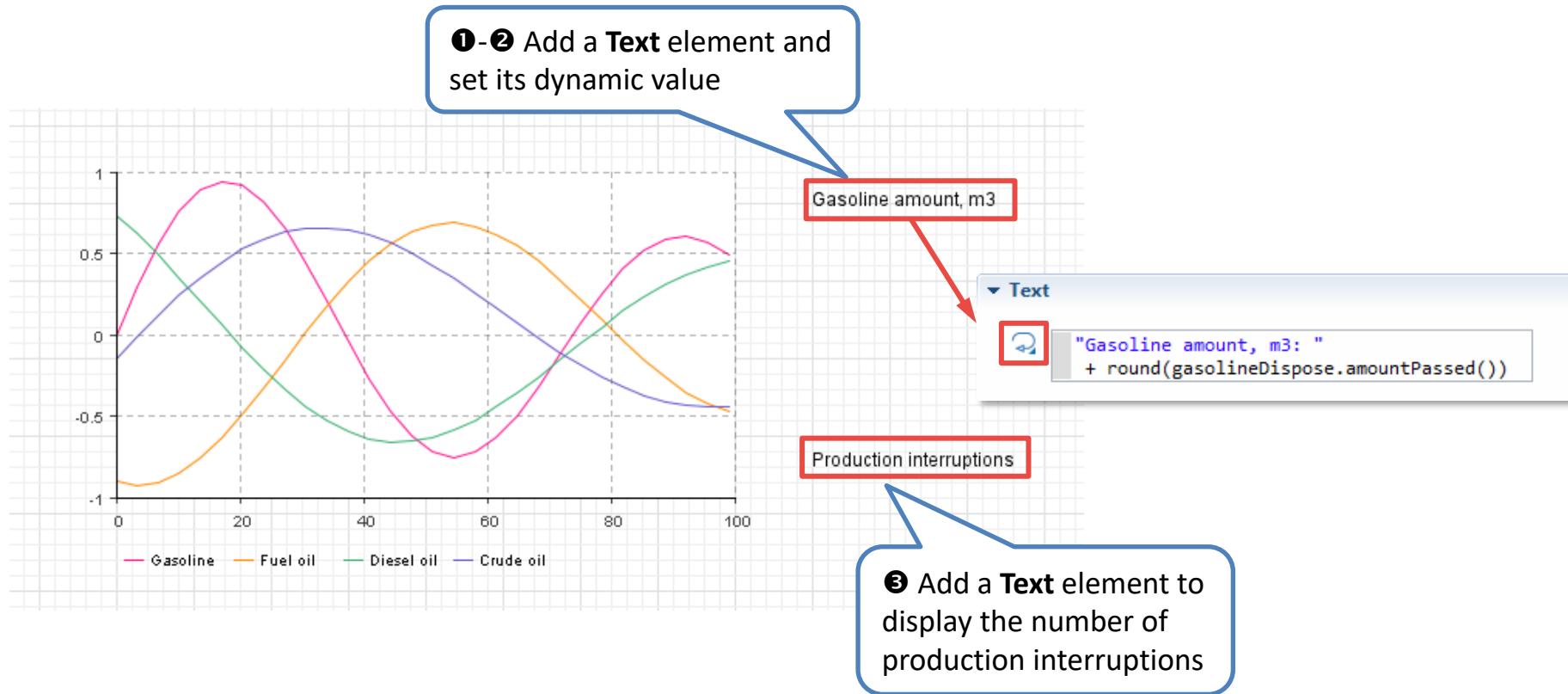
3 Adjust the chart's properties

Now we will create a dashboard to observe the system performance.

- ❶ Drag the **Time Plot** element from the **Analysis** palette onto the **Main** diagram.
- ❷ The chart will display the amount of crude oil and refined products available in the corresponding tanks at each moment of time.
Add a chart data item for each tank by clicking the  **Add** button in the **Data** section of the chart. In the **Title** field of each data item, type the name of the product. In the **Value** field of each data item, provide the call to the *amount()* function of the corresponding **Tank** block, which returns the amount currently contained in the tank.
- ❸ In the **Data update** section, set the chart to **Display up to 2400** latest samples. Since the chart's data is updated every model hour, this will allow us to view the data for 100 days of model time, which is the simulation time horizon that we defined earlier.

In the **Scale** section, set the chart's **Time window** to *100 days*.

Oil Refinery Plant. Phase 5. Step 4



We will use the **Text** elements to display a set of metrics that will give us an overview of the system performance.

- ➊ Add the **Text** element from the **Presentation** palette.
- ➋ Switch the **Text** field to the **Dynamic value** mode by clicking the  button to the left of the field. The icon will change to . Type the following text in the field:

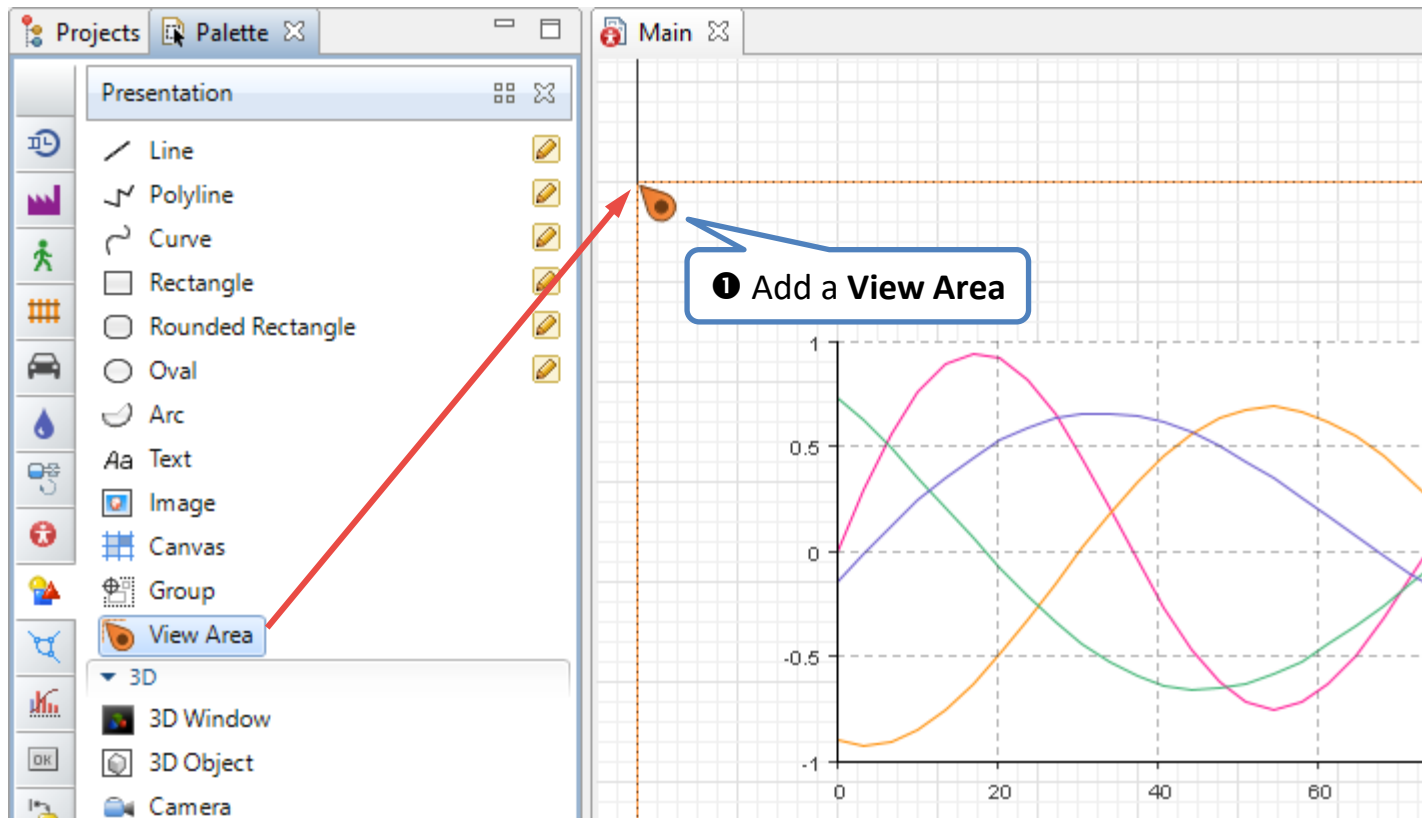
"Gasoline amount, m3: " + round(gasolineDispose.amountPassed())

Here we call the *amountPassed()* function to obtain the amount of gasoline that passed through the corresponding **Fluid Dispose** block. We call *round()* to round the obtained value.

Repeat the ➊-➋ steps for each refined product.

- ➌ Add another text. Set the following text in the **Dynamic value** mode to obtain the number of production interruptions:
"Production interruptions: " + productionInterruptions

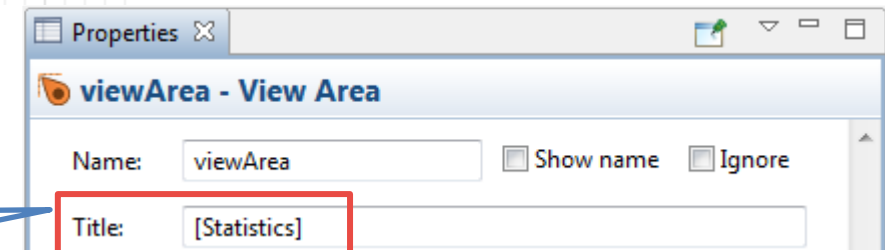
Oil Refinery Plant. Phase 5. Step 5



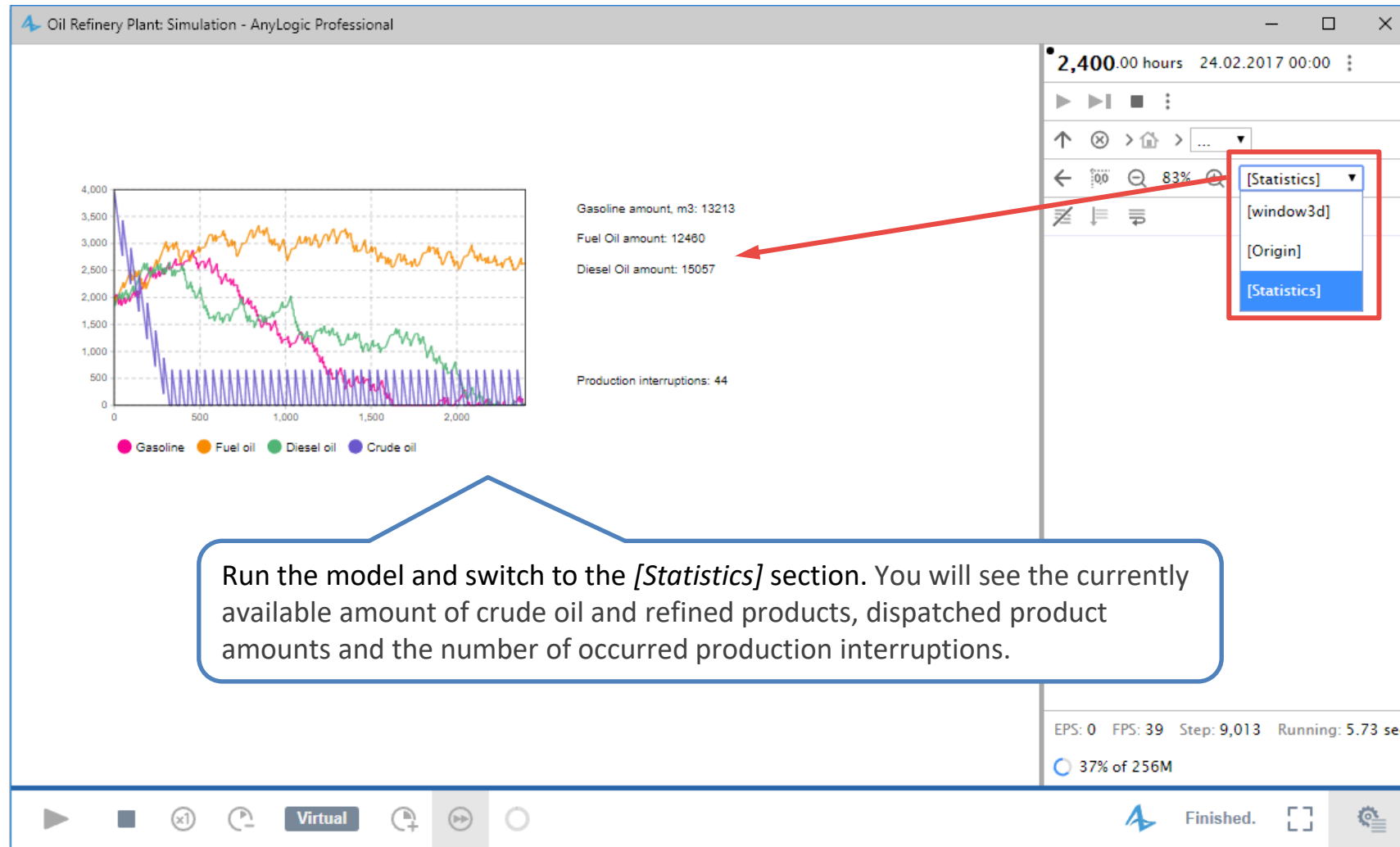
Let us add a **View Area** that will allow us to quickly navigate to the statistics section during the model run.

- 1 Drag the **View Area** element from the **Presentation** palette onto the **Main** diagram and position it above the chart.
- 2 Set the **View Area**'s Title to *[Statistics]*.

2 Set Title to *[Statistics]*



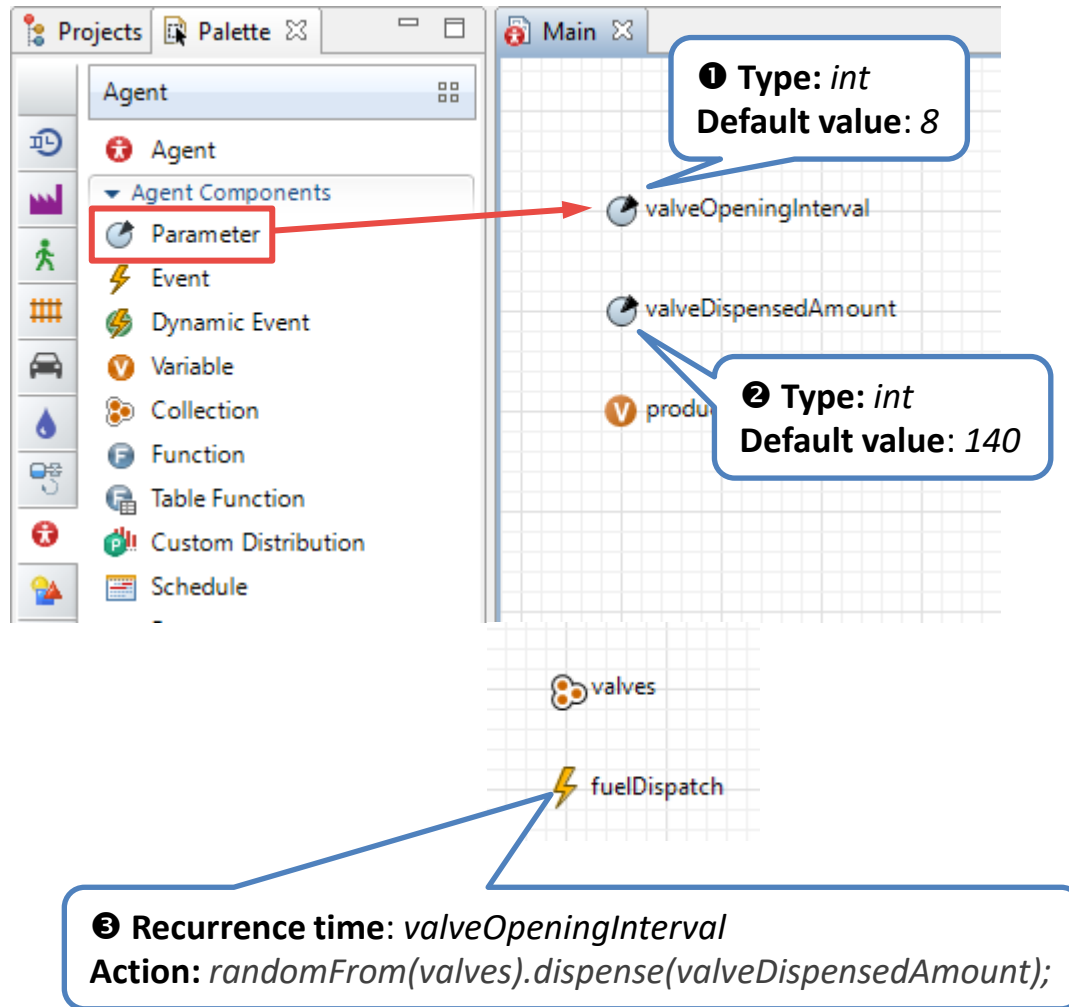
Oil Refinery Plant. Phase 5. Step 6



Oil Refinery Plant. Phase 6.

- In real life, an oil refinery plant operates continuously, and production interruptions must be avoided.
- In this phase, we will create an optimization experiment to find the values of the model's parameters which provide the minimum number of production interruptions.

Oil Refinery Plant. Phase 6. Step 1



We will add several parameters to our model and use them instead of numerical values. We will then vary the parameters' values during the optimization experiment and find the optimal ones.

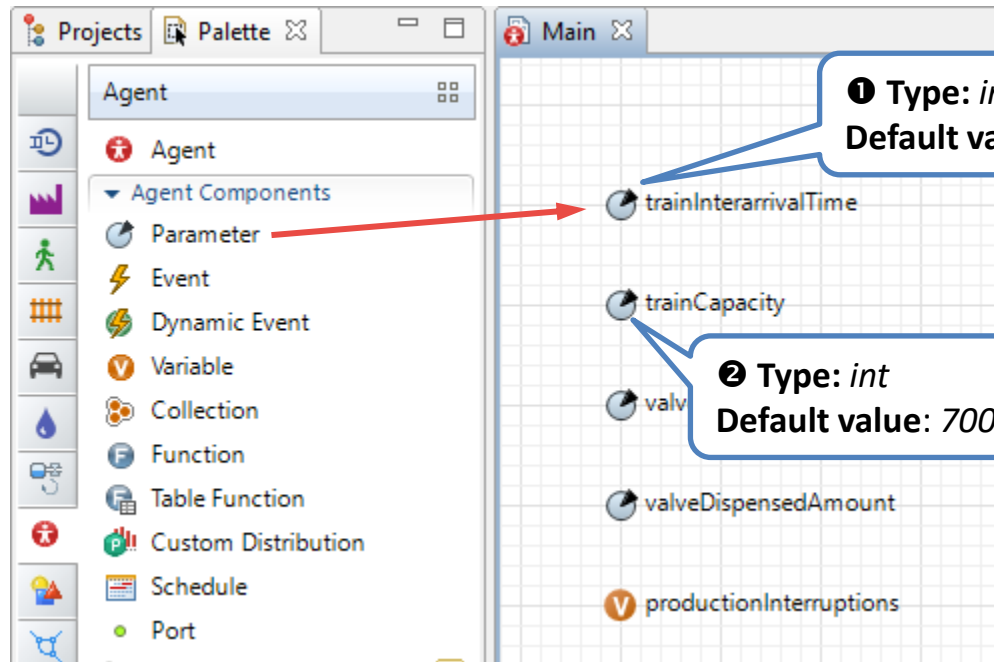
We will start with parameterizing the frequency and amount of oil products dispatches.

①-② Add two parameters. Set their **Type** to *int* and provide their default values.

③ Adjust the *fuelDispatch* event:

- Set the event's **Recurrence time** to be regulated by the *valveOpeningInterval* parameter.
- Set the amount of oil products dispensed through the valves to be regulated by the *valveDispensedAmount* parameter. Adjust the event's **Action** as follows:
randomFrom(valves).dispense(valveDispensedAmount);

Oil Refinery Plant. Phase 6. Step 2

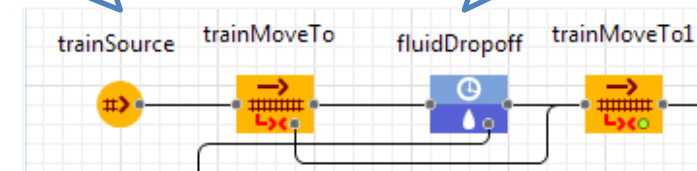


❶ Type: *int*
Default value: 48

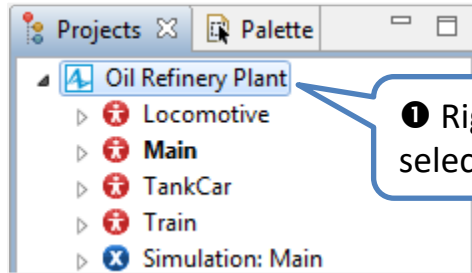
❷ Type: *int*
Default value: 700

❸ *trainSource*
Interarrival time: *trainInterarrivalTime*

❹ *fluidDropoff*
Fluid to dropoff: *trainCapacity*

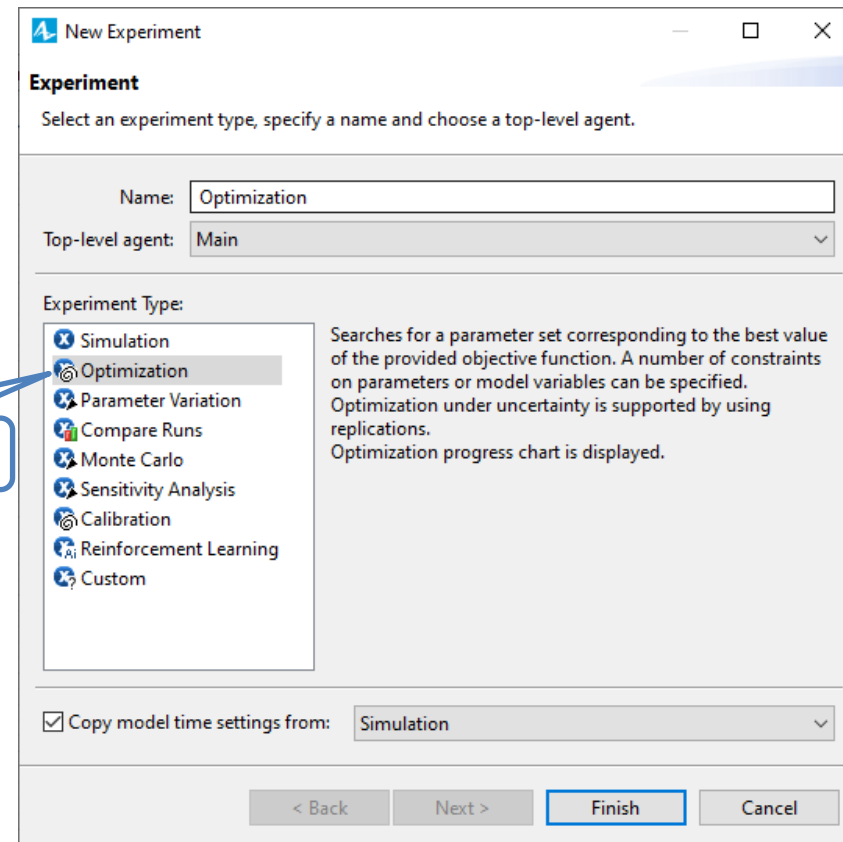


Oil Refinery Plant. Phase 6. Step 3



❶ Right-click the model and select **New > Experiment**

❷ Select **Optimization**



Oil Refinery Plant. Phase 6. Step 4

Now we will configure the experiment.

- 1 Define the objective function, which is the value that we want to minimize as a result of the experiment.
In our case, it is the number of production interruptions, which is provided by the *productionInterruptions* variable that we defined in the top-level agent of the experiment. Since the top-level agent is accessed as *root* in the experiment's code, we type here:
root.productionInterruptions
- 2 Leave the **Number of iterations** option set to **Fixed**, 500.
- 3 Configure the optimization parameters as displayed on the screenshot to vary the trains interarrival time, the amount of crude oil delivered by the train, the oil products dispatches frequency and the amount of dispatched refined products.

Optimization - Optimization Experiment

Name: Optimization ☐ Ignore

Top-level agent: Main

Optimization engine: Genetic

Objective: ☒ minimize ☐ maximize
root.productionInterruptions

Number of iterations
☒ Fixed: 500 ☐ Infinite

Maximum available memory: 256

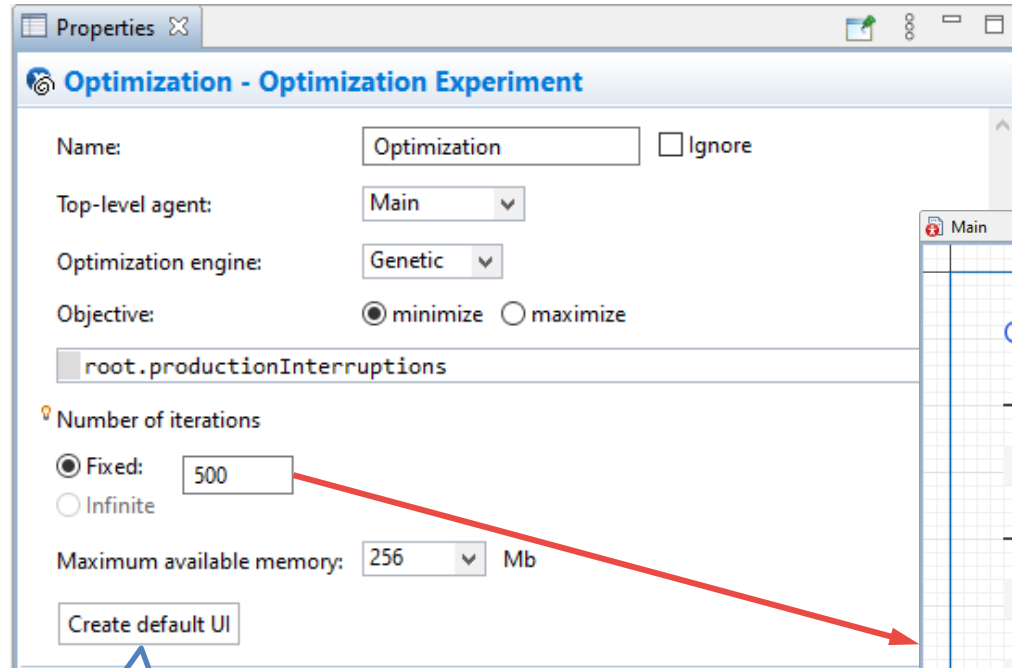
Create default UI

Parameters

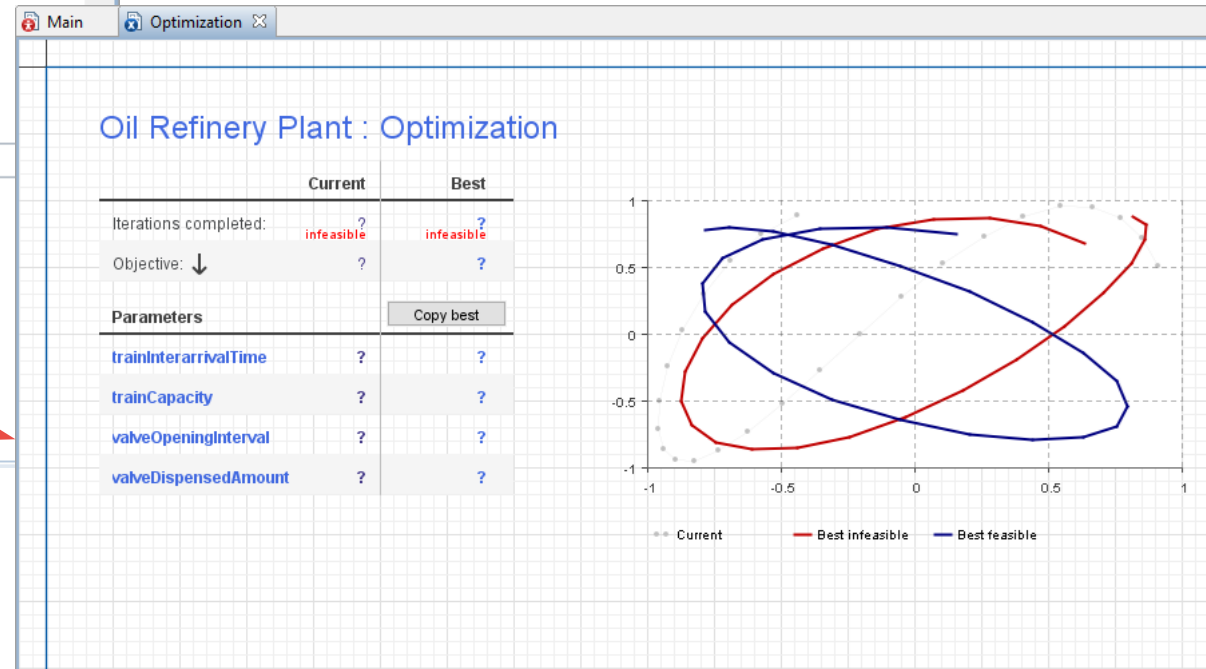
Parameters:

Parameter	Type	Value			
		Min	Max	Step	Suggested
trainInterarrivalTime	discrete	24	96	24	
trainCapacity	discrete	70	700	70	
valveOpeningInterval	discrete	8	24	8	
valveDispensedAmount	discrete	70	350	70	

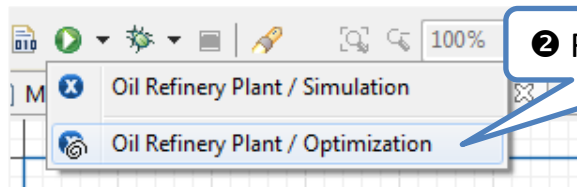
Oil Refinery Plant. Phase 6. Step 5



❶ Click to create the experiment UI



❷ Run the experiment



Oil Refinery Plant. Phase 6. Step 6

The screenshot displays the 'Oil Refinery Plant : Optimization' window in AnyLogic Professional. It features a table comparing 'Current' and 'Best' values for various parameters. A 'Copy best' button is visible next to the table. A scatter plot is also shown in the background. A 'Clipboard access' dialog box is open, asking for permission to copy data to the clipboard. A 'Properties' window for the 'Simulation - Simulation Experiment' is also visible, showing parameters like 'trainInterarrivalTime', 'trainCapacity', 'valveOpeningInterval', and 'valveDispensedAmount' with their respective values. A 'Paste from clipboard' button is located at the bottom of the Properties window.

Oil Refinery Plant : Optimization

	Current	Best
Iterations completed:	225	16
Objective: ↓	86	0
Parameters		Copy best
trainInterarrivalTime	72	24
trainCapacity	210	700
valveOpeningInterval	24	8
valveDispensedAmount	350	350

Clipboard access

The model wants to copy data to the clipboard.
[Allow](#) or close this dialog to reject.

Simulation - Simulation Experiment

Name: Simulation ☐ Ignore

Top-level agent: Main

Maximum available memory: 256 Mb

☐ Skip experiment screen and run the model

Parameters

trainInterarrivalTime: 24

trainCapacity: 700

valveOpeningInterval: 8

valveDispensedAmount: 350

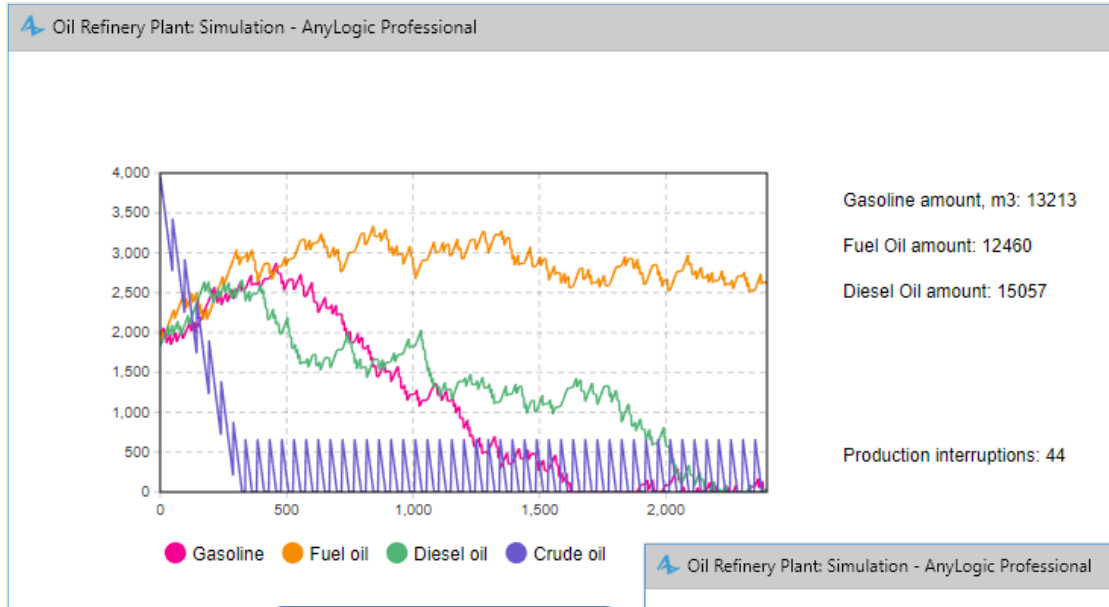
Paste from clipboard

Step 1: Click to copy the found optimal parameters' values to the clipboard

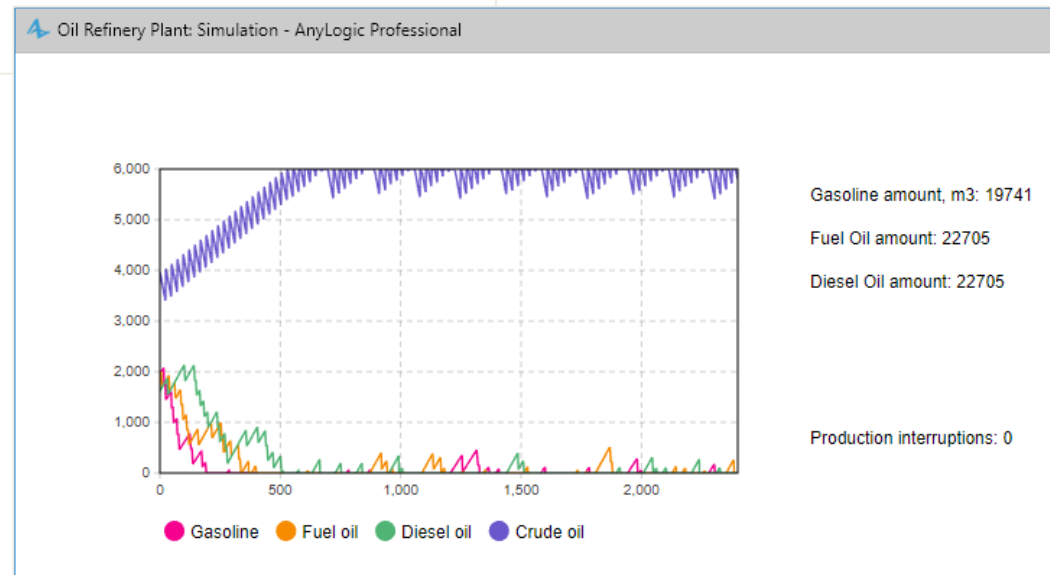
Step 2: Allow clipboard access to enable copying values

Step 3: Paste the found optimal parameters' values

Oil Refinery Plant. Phase 6. Step 7



Run the model and
compare the results



Run the **Simulation** experiment again and compare the results.

As you can see, no production interruptions have occurred in our model, as opposed to 44 interruptions before optimization, while the amount of dispatched products has increased significantly.